



香港科技大學
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

ELEC 3310

Digital Fundamentals and System Design

Wenchao Miao

Outline Lecture 2 Combinational Logic



THE HONG KONG
UNIVERSITY OF SCIENCE
AND TECHNOLOGY

1 Introduction & Learning Outcomes

2 Number Systems

3 Boolean Algebra and Logic Simplification

4 Combinational Logic Analysis

5 Functions of Combinational Logic

6 Logic Functions & Gates

7 Boolean Algebra

1. Course focus (Numbers, Adders, Boolean Expressions)

1. Convert from decimal or hexadecimal to binary and from binary to decimal or hexadecimal
2. Develop a truth table for a combinational logic circuit based on the intended behavior (specification)
3. Derive the Boolean expression from a truth table in terms of sum of product terms (minterms) or product of sum terms (maxterms)
4. Describe a basic procedure for combinational logic design
5. Design simple combinational logic circuits including half adder, full adder, and magnitude comparator using the prescribed procedure
6. Use full adders to implement multi-bit parallel (ripple) binary adders
7. Explain the difference between ripple carry and look-ahead carry parallel adders

Main Content

1. Decimal & Binary Fundamentals
2. Hexadecimal & Octal
3. Conversions

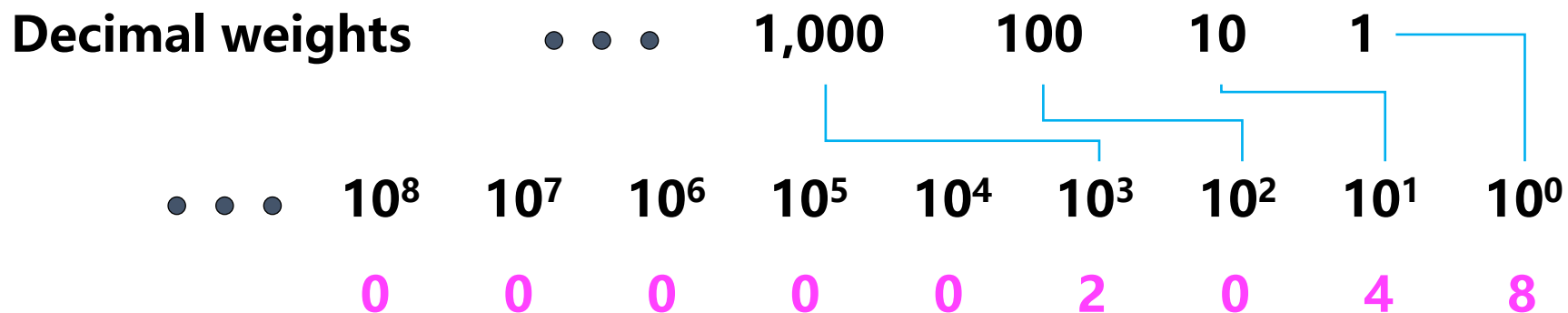
2. Number Systems

1. Decimal & Binary Fundamentals

(1) Decimal system (base-10, whole numbers)

- Ten decimal digits because we have ten fingers
 - 0, 1, 2, 3, 4, 5, 6, 7, 8, and 9
 - After 9, increase number of digits: 10, 11, 12, ..., 19, 20,.. 99, 100, 101
- The decimal numbering system has a base of 10, meaning each digit position being weighted by a factor of 10

Example: decimal number 2048

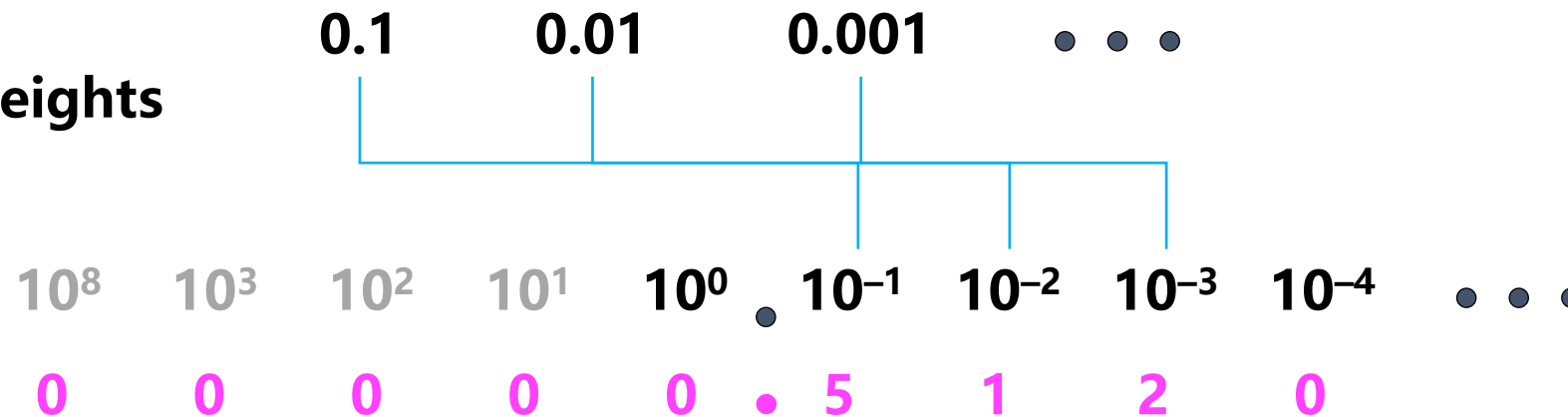


2. Number Systems

(2) Fractional decimal numbers

Example: decimal number 0.512

Fractional
decimal weights



- All the digits to the left of the decimal point have weights that are positive powers of ten
- All the digits to the right of the decimal point have weights that are negative powers of ten, or fractional weights

(3) Binary system (base-2, weights)

- Two binary digits (bits)
 - 0 and 1
 - After 1, increase number of bits: 10, 11, 100, 101, 110, 111, 1000
- The binary system has a base of 2, meaning each digit position being weighted by a factor of 2

Binary weights.

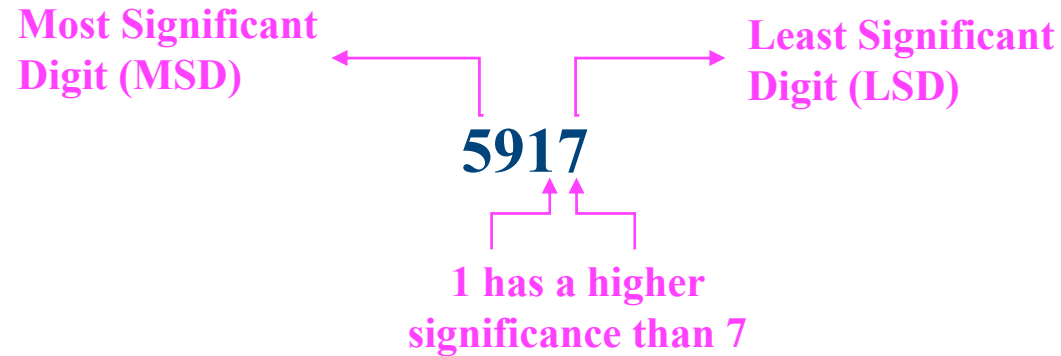
Positive Powers of Two (Whole Numbers)									Negative Powers of Two (Fractional Number)					
2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	2^{-1}	2^{-2}	2^{-3}	2^{-4}	2^{-5}	2^{-6}
256	128	64	32	16	8	4	2	1	1/2 0.5	1/4 0.25	1/8 0.125	1/16 0.0625	1/32 0.03125	1/64 0.015625

- All the bits to the left of the binary point have weights that are positive powers of 2
- All the bits to the right of the binary point have weights that are negative powers of 2, or fractional weights

(3) Binary system (base-2, weights)

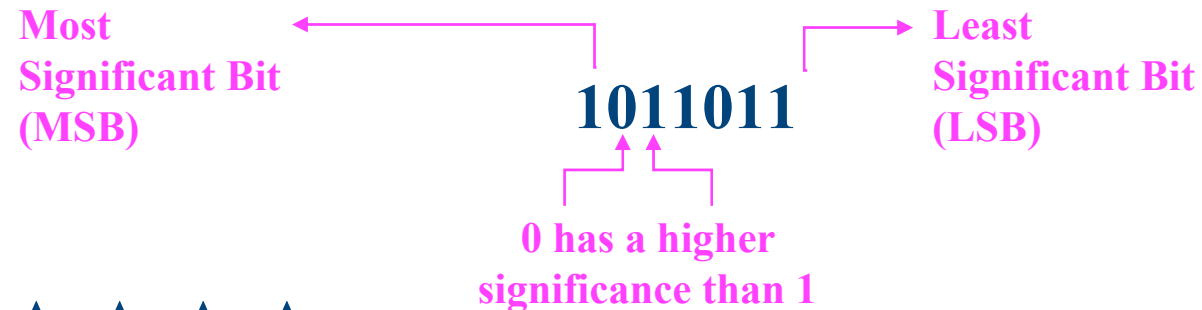
- Decimal

Example:



- Binary

Example:



4-bit number: $A_3 A_2 A_1 A_0$



A_i : the larger the index i the greater the significance of the bit A_i in the binary number

2. Number Systems

(4) 4-bit binary number range (0–15)

Decimal Number	2^3	2^2	2^1	2^0
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1
10	1	0	1	0
11	1	0	1	1
12	1	1	0	0
13	1	1	0	1
14	1	1	1	0
15	1	1	1	1

4-bit binary numbers can represent 16 decimal numbers (from 0 to 15)

Maximum number that can be represented by a 4-bit number: 15

4 because there are 4 bits

$$2^4 - 1 = 15$$

2 because each bit can be 0 or 1

Subtract 1 for decimal 0

(5) An Application

A simple example of counting tennis balls going into a box from a conveyor belt

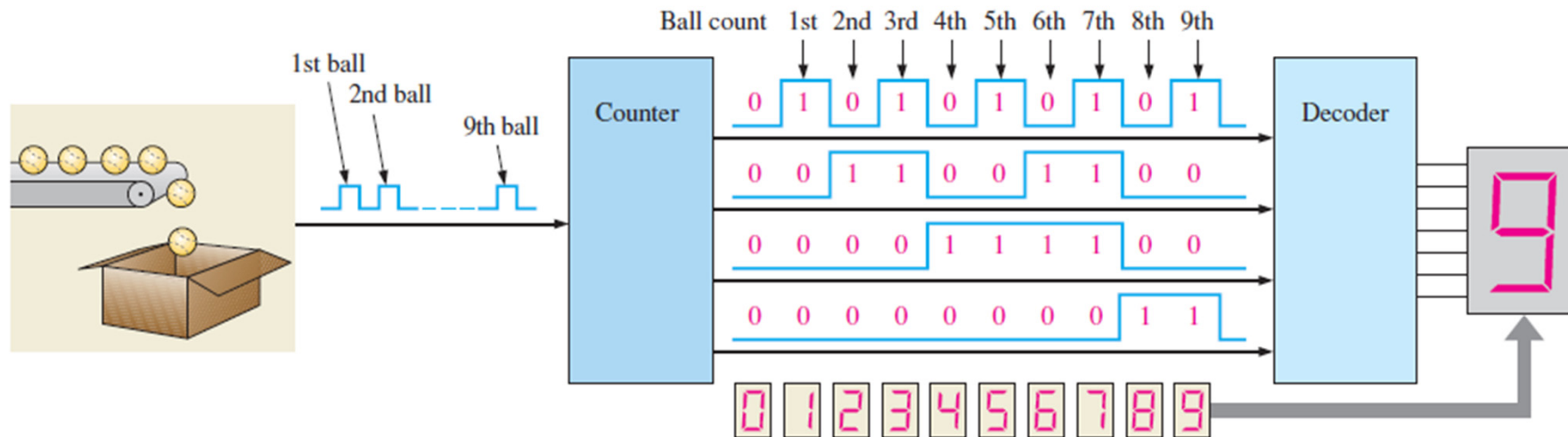


Illustration of a simple binary counting application

2. Hexadecimal & Octal

(1) Number representations in bases 2, 8, 10, 16

Decimal (base 10)	Binary (base 2)	Octal (base 8)	Hexadecimal (base 16)
00	0000	00	0
01	0001	01	1
02	0010	02	2
03	0011	03	3
04	0100	04	4
05	0101	05	5
06	0110	06	6
07	0111	07	7
08	1000	10	8
09	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

2. Number Systems

3. Conversions

(1) Decimal $\leftrightarrow$ Binary (whole & fractional)

Example: decimal number, 103 or $(103)_{10}$

- Start from MSB and place 0 or 1 depending on whether to include the weight of that bit
- Move to lower bits and repeat the procedure until all the weights add up to decimal number
- Goal is to obtain a given decimal number by including binary weights as large as possible

Binary weights.

Positive Powers of Two (Whole Numbers)									Negative Powers of Two (Fractional Number)					
2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	2^{-1}	2^{-2}	2^{-3}	2^{-4}	2^{-5}	2^{-6}
256	128	64	32	16	8	4	2	1	1/2 0.5	1/4 0.25	1/8 0.125	1/16 0.0625	1/32 0.03125	1/64 0.015625

0 0 1 1 0 0 1 1 1

$$103 - 64 = 39$$

$$39 - 32 = 7$$

$$(103)_{10} = (1100111)_2$$

2. Number Systems

(1) Decimal $\leftrightarrow$ Binary (whole & fractional)

More examples

2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	=
256	128	64	32	16	8	4	2	1	=
0	0	1	0	0	1	0	0	0	72
0	0	0	1	0	0	0	0	1	33
0	1	0	1	0	0	1	1	1	167

2. Number Systems

(1) Decimal $\leftrightarrow$ Binary (whole & fractional)

Example: fractional decimal number, 5.1875 or $(5.1875)_{10}$

- Convert whole part and fractional part separately
- Convert whole part of the number as describe before
- Convert the fractional part of the number as described before but start from the bit to the right of binary point

Binary weights.

Positive Powers of Two (Whole Numbers)									Negative Powers of Two (Fractional Number)					
2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	2^{-1}	2^{-2}	2^{-3}	2^{-4}	2^{-5}	2^{-6}
256	128	64	32	16	8	4	2	1	1/2 0.5	1/4 0.25	1/8 0.125	1/16 0.0625	1/32 0.03125	1/64 0.015625
0	0	0	0	0	0	1	0	1	• 0	0	1	1	0	0

$$0.1875 - 0.125 = 0.0625$$

$$(5.1875)_{10} = (101.0011)_2$$

2. Number Systems

(1) Decimal $\leftrightarrow$ Binary (whole & fractional)

More examples

Whole part				Fractional part				
8	4	2	1	1/2	1/4	1/8	1/16	=
1	0	1	1	0	1	1	0	11.375
0	0	0	0	0	1	1	1	0.4375
0	0	1	1	0	0	0	1	3.0625

2. Number Systems

(1) Decimal $\leftrightarrow$ Binary (whole & fractional)

Example: binary number, 10110110 or $(10110110)_2$

- Add binary weights of all bits that are 1
- Same for whole and fractional parts

Binary weights.

Positive Powers of Two (Whole Numbers)									Negative Powers of Two (Fractional Number)					
2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	2^{-1}	2^{-2}	2^{-3}	2^{-4}	2^{-5}	2^{-6}
256	128	64	32	16	8	4	2	1	1/2	1/4	1/8	1/16	1/32	1/64
									0.5	0.25	0.125	0.0625	0.03125	0.015625

1 0 1 1 0 1 1 0

$$128 + 32 + 16 + 4 + 2 = 182$$

$$(10110110)_2 = (182)_{10}$$

2. Number Systems

(1) Decimal $\leftrightarrow$ Binary (whole & fractional)

Example: fractional binary number, 0.1101 or $(0.1101)_2$

- Add binary weights of all bits that are 1
- Same for whole and fractional parts

Binary weights.

Positive Powers of Two (Whole Numbers)									Negative Powers of Two (Fractional Number)					
2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	2^{-1}	2^{-2}	2^{-3}	2^{-4}	2^{-5}	2^{-6}
256	128	64	32	16	8	4	2	1	1/2	1/4	1/8	1/16	1/32	1/64
									0.5	0.25	0.125	0.0625	0.03125	0.015625

0 . 1 1 0 1 0 0

$$0.5 + 0.25 + 0.0625 = 0.8125$$

$$(0.1101)_2 = (0.8125)_{10}$$

(2) Binary $\leftrightarrow$ Hexadecimal

Binary-to-Hexadecimal Conversion

Converting a binary number to hexadecimal is a straightforward procedure. Simply break the binary number into 4-bit groups, starting at the right-most bit and replace each 4-bit group with the equivalent hexadecimal symbol.

EXAMPLE 2-24

Convert the following binary numbers to hexadecimal:

- (a) 1100101001010111 (b) 111111000101101001

Solution

$$\begin{array}{lcl} \text{(a)} & \begin{array}{cccc} 1100 & 1010 & 0101 & 0111 \\ \downarrow & \downarrow & \downarrow & \downarrow \\ C & A & 5 & 7 \end{array} & = CA57_{16} \\ \text{(b)} & \begin{array}{ccccc} 0011 & 1111 & 0001 & 0110 & 1001 \\ \downarrow & \downarrow & \downarrow & \downarrow & \downarrow \\ 3 & F & 1 & 6 & 9 \end{array} & = 3F169_{16} \end{array}$$

Two zeros have been added in part (b) to complete a 4-bit group at the left.

Related Problem $\begin{array}{ccccccc} & 4 & F & 7 & 9 & C & \\ & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \\ 0 & 1001 & 1110 & 1110 & 0111 & 00 & \end{array} = 4F79C_{16}$
Convert the binary number 01001111011110011100 to hexadecimal.

(3) Hexadecimal $\leftrightarrow$ Binary

Hexadecimal-to-Binary Conversion

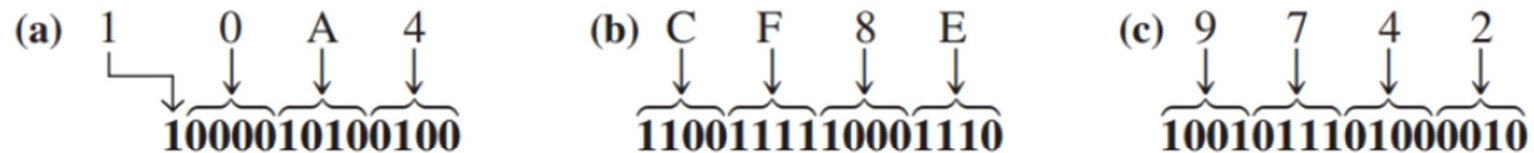
To convert from a hexadecimal number to a binary number, reverse the process and replace each hexadecimal symbol with the appropriate four bits.

EXAMPLE 2-25

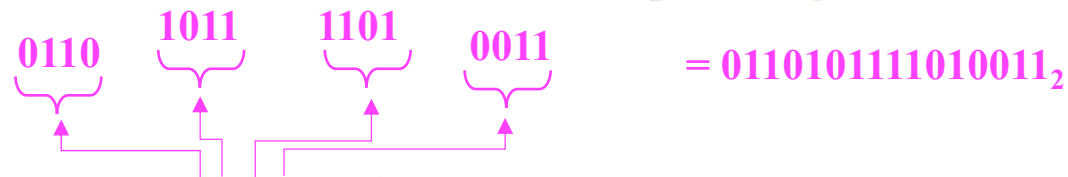
Determine the binary numbers for the following hexadecimal numbers:

- (a) $10A4_{16}$ (b) $CF8E_{16}$ (c) 9742_{16}

Solution



In part (a), the MSB is understood to have three zeros preceding it, thus forming a 4-bit group.



Related Problem

Convert the hexadecimal number 6BD3 to binary.



Main Content

1. Boolean Analysis of Logic Circuits
2. Logic Simplification Using Boolean Algebra
3. Standard Forms of Boolean Expressions
4. Boolean Expressions and Truth Tables
5. The Karnaugh Map
6. Karnaugh Map SOP Minimization
7. Karnaugh Map POS Minimization
8. The Quine-McCluskey Method

1. Boolean Analysis of Logic Circuits

To derive the Boolean expression for a given combinational logic circuit, begin at the left-most inputs and work toward the final output, writing the expression for each gate

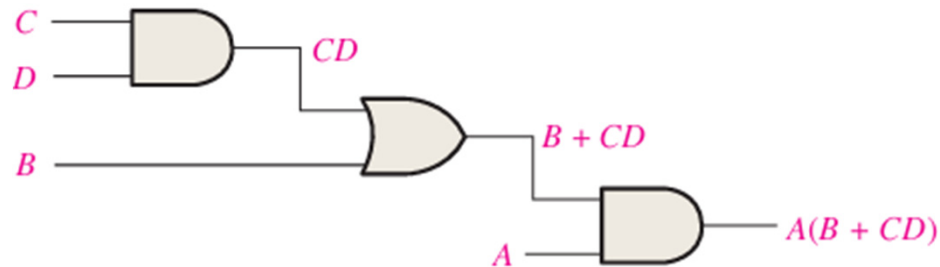


FIGURE 4–18 A combinational logic circuit showing the development of the Boolean expression for the output

TABLE 4–5

Truth table for the logic circuit in Figure 4–18.

Inputs				Output
<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	$A(B + CD)$
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

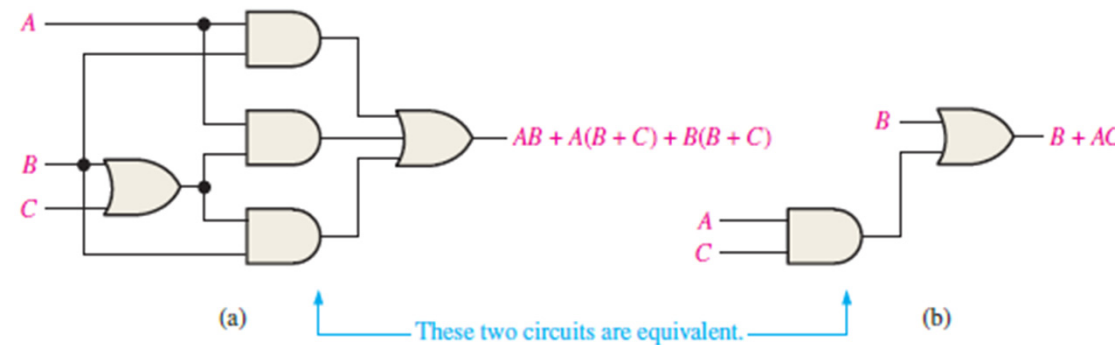
2. Logic Simplification Using Boolean Algebra

A logic expression can be reduced to its simplest form or changed to a more convenient form to implement the expression most efficiently using Boolean algebra

Example

Using Boolean algebra techniques, simplify this expression:

$$AB + A(B + C) + B(B + C)$$



2. Logic Simplification Using Boolean Algebra

Solution

The following is not necessarily the only approach

Step 1: Apply the distributive law to the second and third terms in the expression, as follows:

$$AB + AB + AC + BB + BC$$

Step 2: Apply rule 7 ($BB = B$) to the fourth term

$$AB + AB + AC + B + BC$$

Step 3: Apply rule 5 ($AB + AB = AB$) to the first two terms

$$AB + AC + B + BC$$

Step 4: Apply rule 10 ($B + BC = B$) to the last two terms

$$AB + AC + B$$

Step 5: Apply rule 10 ($AB + B = B$) to the first and third terms

$$B + AC$$

At this point the expression is simplified as much as possible. Once you gain experience in applying Boolean algebra, you can often combine many individual steps

3. Standard Forms of Boolean Expressions

All Boolean expressions, regardless of their form, can be converted into either of two standard forms: the sum-of-products form or the product-of-sums form

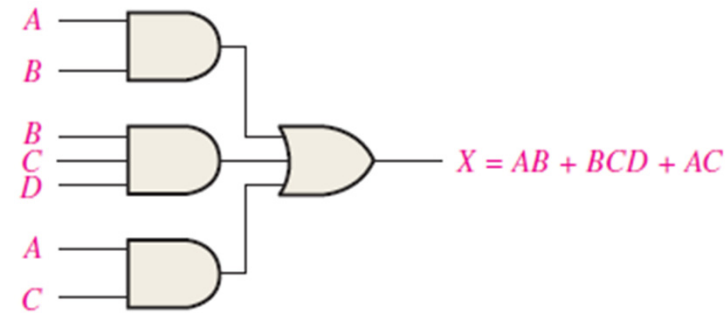
The Sum-of-Products (SOP) Form

A product term was defined before as a term consisting of the product (Boolean multiplication) of literals (variables or their complements). When two or more product terms are summed by Boolean addition, the resulting expression is a **sum-of-products (SOP)**

$$AB + ABC$$

$$ABC + CDE + \overline{BCD}$$

$$\overline{AB} + \overline{ABC} + AC$$



Implementation of the SOP expression $AB + BCD + AC$

3. Standard Forms of Boolean Expressions

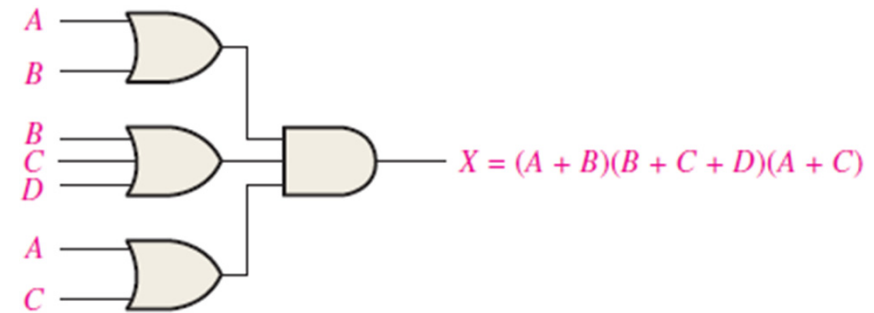
The Product-of-Sums (POS) Form

A sum term was defined in Section 4–1 as a term consisting of the sum (Boolean addition) of literals (variables or their complements). When two or more sum terms are multiplied, the resulting expression is a **product-of-sums (POS)**. Some examples are

$$(\bar{A} + B)(A + \bar{B} + C)$$

$$(\bar{A} + \bar{B} + \bar{C})(C + \bar{D} + E)(\bar{B} + C + D)$$

$$(A + B)(A + \bar{B} + C)(\bar{A} + C)$$



4. Boolean Expressions and Truth Tables

All standard Boolean expressions can be easily converted into truth table format using binary values for each term in the expression

Example

Develop a truth table for the standard SOP expression $\overline{A}\overline{B}C + A\overline{B}\overline{C} + ABC$

TABLE 4-6				
Inputs			Output	Product Term
<i>A</i>	<i>B</i>	<i>C</i>	<i>X</i>	
0	0	0	0	
0	0	1	1	$\overline{A}\overline{B}C$
0	1	0	0	
0	1	1	0	
1	0	0	1	$A\overline{B}\overline{C}$
1	0	1	0	
1	1	0	0	
1	1	1	1	ABC

5. The Karnaugh Map

- A Karnaugh map provides a systematic method for simplifying Boolean expressions and, if properly used, will produce the simplest SOP or POS expression possible, known as the minimum expression
- Other simplification techniques include the Quine-McCluskey method and the Espresso algorithm

AB \ C	C	
	0	1
00		
01		
11		
10		

(a)

AB \ C	C	
	0	1
00	$\bar{A}\bar{B}\bar{C}$	$\bar{A}\bar{B}C$
01	$\bar{A}B\bar{C}$	$\bar{A}BC$
11	$AB\bar{C}$	ABC
10	$A\bar{B}\bar{C}$	$A\bar{B}C$

(b)

A 3-variable Karnaugh map showing Boolean product terms for each cell

AB \ CD	CD			
	00	01	11	10
00				
01				
11				
10				

(a)

AB \ CD	CD			
	00	01	11	10
00	$\bar{A}\bar{B}\bar{C}\bar{D}$	$\bar{A}\bar{B}\bar{C}D$	$\bar{A}\bar{B}C\bar{D}$	$\bar{A}\bar{B}CD$
01	$\bar{A}B\bar{C}\bar{D}$	$\bar{A}B\bar{C}D$	$\bar{A}BC\bar{D}$	$\bar{A}BCD$
11	$AB\bar{C}\bar{D}$	$AB\bar{C}D$	$ABC\bar{D}$	$ABCD$
10	$A\bar{B}\bar{C}\bar{D}$	$A\bar{B}\bar{C}D$	$A\bar{B}C\bar{D}$	$A\bar{B}CD$

(b)

A 4-variable Karnaugh map

AB \ CD	CD			
	00	01	11	10
00				
01				
11				
10				

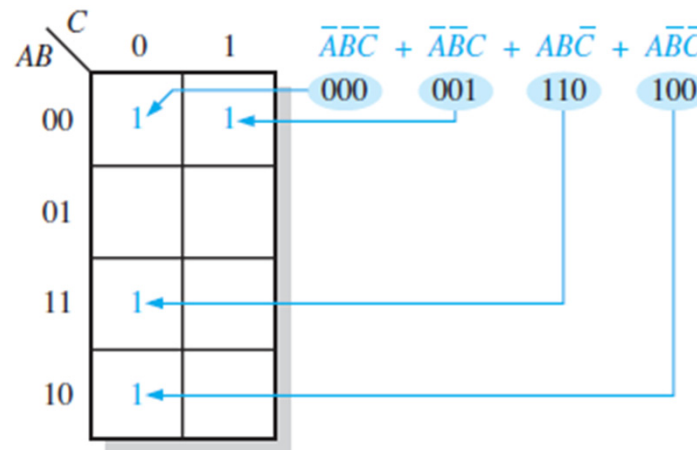
Adjacent cells on a Karnaugh map are those that differ by only one variable. Arrows point between adjacent cells

6. Karnaugh Map SOP Minimization

- A minimized SOP expression contains the fewest possible terms with the fewest possible variables per term
- Generally, a minimum SOP expression can be implemented with fewer logic gates than a standard expression

Mapping a Standard SOP Expression

For an SOP expression in standard form, a 1 is placed on the Karnaugh map for each product term in the expression



Example of mapping a standard SOP expression

6. Karnaugh Map SOP Minimization

Karnaugh Map Simplification of SOP Expressions

The process that results in an expression containing the fewest possible terms with the fewest possible variables is called **minimization**. After an SOP expression has been mapped, a minimum SOP expression is obtained by grouping the 1s and determining the minimum SOP expression from the map

Grouping the 1s

You can group 1s on the Karnaugh map according to the following rules by enclosing those adjacent cells containing 1s. The goal is to maximize the size of the groups and to minimize the number of groups

Group the 1s in each of the Karnaugh maps in Figure

(a)

AB \ C	0	1
	00	01
00	1	
01		1
11	1	1
10		

(b)

AB \ C	0	1
	00	01
00	1	1
01	1	
11		1
10	1	1

(c)

AB \ CD	00	01	11	10
	00	01	11	10
00	1	1		
01	1	1	1	1
11				
10		1	1	

(d)

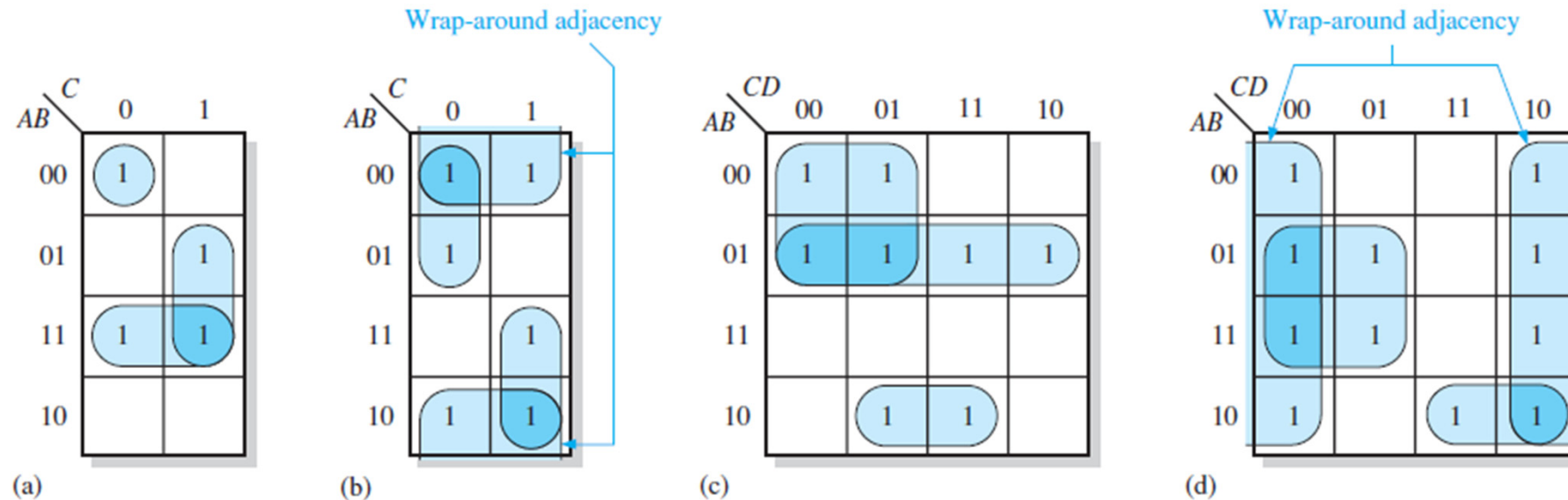
AB \ CD	00	01	11	10
	00	01	11	10
00	1			1
01	1	1		1
11	1	1		1
10	1		1	1



6. Karnaugh Map SOP Minimization

Solution

The groupings are shown in Figure. In some cases, there may be more than one way to group the 1s to form maximum groupings



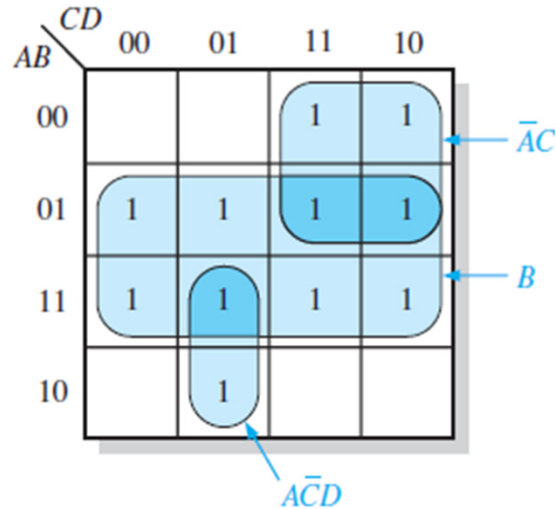


6. Karnaugh Map SOP Minimization

Determining the Minimum SOP Expression from the Map

When all the 1s representing the standard product terms in an expression are properly mapped and grouped, the process of determining the resulting minimum SOP expression begins

Determine the product terms for the Karnaugh map in Figure and write the resulting minimum SOP expression



$$B + \bar{A}C + A\bar{C}D$$

7. Karnaugh Map POS Minimization

We focus on POS expressions. The approaches are much the same except that with POS expressions, 0s representing the standard sum terms are placed on the Karnaugh map instead of 1s

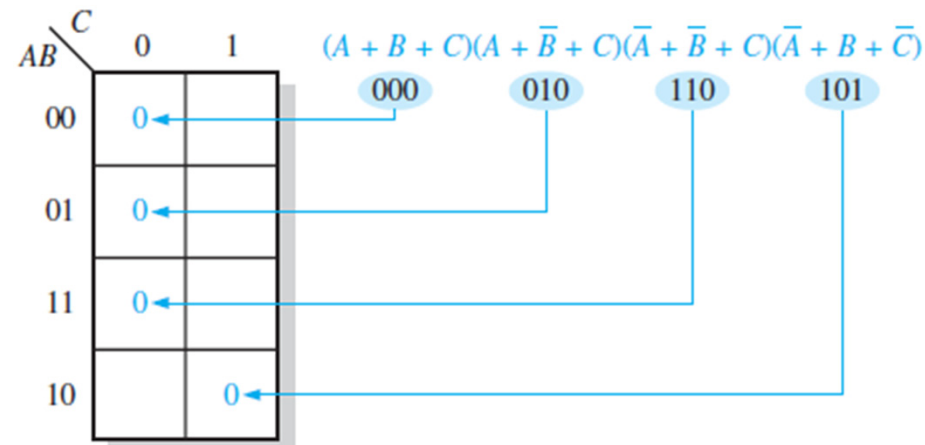
Mapping a Standard POS Expression

For a POS expression in standard form, a 0 is placed on the Karnaugh map for each sum term in the expression. Each 0 is placed in a cell corresponding to the value of a sum term

Step 1: Determine the binary value of each sum term in the standard POS expression.

This is the binary value that makes the term equal to 0.

Step 2: As each sum term is evaluated, place a 0 on the Karnaugh map in the corresponding cell.



Example of mapping a standard POS expression



8. The Quine-McCluskey Method

The Quine-McCluskey method is a formal tabular method for applying the Boolean distributive law to various terms to find the minimum sum of products by eliminating literals that appear in two terms as complements

To apply the Quine-McCluskey method, first write the function in standard **minterm** (SOP) form

TABLE 4-9

<i>ABCD</i>	<i>X</i>	Minterm
0000	0	
0001	1	m_1
0010	0	
0011	1	m_3
0100	1	m_4
0101	1	m_5
0110	0	
0111	0	
1000	0	
1001	0	
1010	1	m_{10}
1011	0	
1100	1	m_{12}
1101	1	m_{13}
1110	0	
1111	1	m_{15}

$$X = \overline{A}\overline{B}\overline{C}\overline{D} + \overline{A}\overline{B}C\overline{D} + \overline{A}B\overline{C}\overline{D} + \overline{A}B\overline{C}D + \overline{A}BC\overline{D} + \overline{A}BCD + A\overline{B}\overline{C}\overline{D} + ABCD$$

TABLE 4-10

Number of 1s	Minterm	<i>ABCD</i>
1	m_1	0001
	m_4	0100
2	m_3	0011
	m_5	0101
	m_{10}	1010
	m_{12}	1100
3	m_{13}	1101
4	m_{15}	1111



8. The Quine-McCluskey Method

TABLE 4-11

Number of 1s in Minterm	Minterm	ABCD	First Level
1	m_1	0001 ✓	(m_1, m_3) 00x1
	m_4	0100 ✓	(m_1, m_5) 0x01
2	m_3	0011 ✓	(m_4, m_5) 010x
	m_5	0101 ✓	(m_4, m_{12}) x100
	m_{10}	1010	(m_5, m_{13}) x101
	m_{12}	1100 ✓	(m_{12}, m_{13}) 110x
3	m_{13}	1101 ✓	(m_{13}, m_{15}) 11x1
4	m_{15}	1111 ✓	

TABLE 4-12

First Level	Number of 1s in First Level	Second Level
(m_1, m_3) 00x1 (m_1, m_5) 0x01 (m_4, m_5) 010x ✓ (m_4, m_{12}) x100 ✓	1	$(m_4, m_5, m_{12}, m_{13})$ x10x $(m_4, m_5, m_{12}, m_{13})$ x10x
(m_5, m_{13}) x101 ✓ (m_{12}, m_{13}) 110x ✓	2	
(m_{13}, m_{15}) 11x1	3	

$$X = \overline{B}\overline{C} + \overline{A}\overline{B}D + \overline{A}\overline{C}D + ABD + A\overline{B}C\overline{D}$$



8. The Quine-McCluskey Method

TABLE 4-13

Prime Implicants	Minterms							
	m_1	m_3	m_4	m_5	m_{10}	m_{12}	m_{13}	m_{15}
$B\bar{C} (m_4, m_5, m_{12}, m_{13})$			✓	✓		✓	✓	
$\bar{A}\bar{B}D (m_1, m_3)$	✓	✓						
$\bar{A}\bar{C}D (m_1, m_5)$	✓			✓				
$ABD (m_{13}, m_{15})$							✓	✓
$\bar{A}\bar{B}\bar{C}\bar{D} (m_{10})$					✓			

$$X = B\bar{C} + \bar{A}\bar{B}D + ABD + \bar{A}\bar{B}\bar{C}\bar{D}$$

Main Content

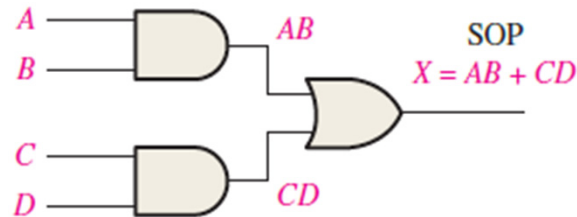
1. Basic Combinational Logic Circuits
2. Implementing Combinational Logic
3. The Universal Property of NAND and NOR Gates

1. Basic Combinational Logic Circuits

We learned that SOP expressions are implemented with an AND gate for each product term and one OR gate for summing all of the product terms. As you know, this SOP implementation is called AND-OR logic and is the basic form for realizing standard Boolean functions

AND-OR Logic

Figure shows an AND-OR circuit consisting of two 2-input AND gates and one 2-input OR gate



Logic diagram (ANSI standard distinctive shape symbols)

TABLE 5-1

Truth table for the AND-OR logic in Figure 5-1.

Inputs						Output
A	B	C	D	AB	CD	X
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	0	1	0	0	0	0
0	0	1	1	0	1	1
0	1	0	0	0	0	0
0	1	0	1	0	0	0
0	1	1	0	0	0	0
0	1	1	1	0	1	1
1	0	0	0	0	0	0
1	0	0	1	0	0	0
1	0	1	0	0	0	0
1	0	1	1	0	1	1
1	1	0	0	1	0	1
1	1	0	1	1	0	1
1	1	1	0	1	0	1
1	1	1	1	1	1	1

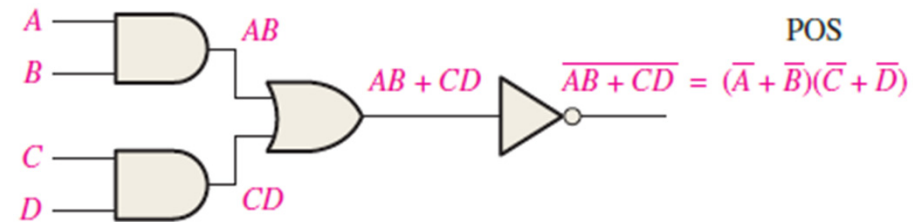
4. Combinational Logic Analysis

1. Basic Combinational Logic Circuits

AND-OR-Invert Logic

When the output of an AND-OR circuit is complemented (inverted), it results in an AND-OR Invert circuit

$$X = (\bar{A} + \bar{B})(\bar{C} + \bar{D}) = (\overline{AB})(\overline{CD}) = \overline{\overline{\overline{AB}}(\overline{CD})} = \overline{\overline{AB} + \overline{CD}} = \overline{AB + CD}$$



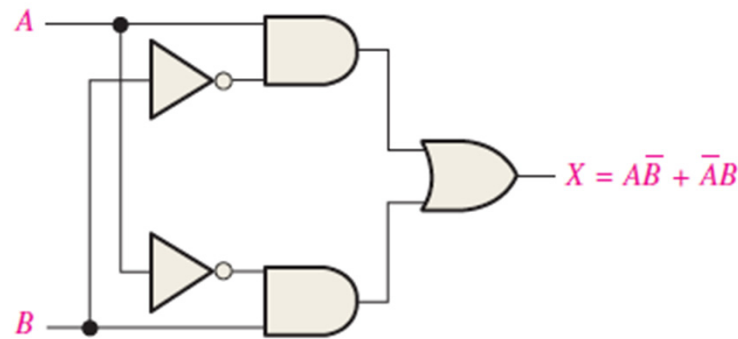
For a 4-input AND-OR-Invert logic circuit, the output X is LOW (0) if both input A and input B are HIGH (1) or both input C and input D are HIGH (1)

A truth table can be developed from the AND-OR truth table by simply changing all 1s to 0s and all 0s to 1s in the output column

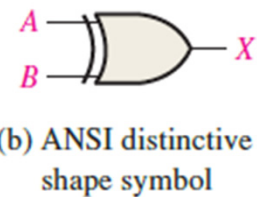
1. Basic Combinational Logic Circuits

Exclusive-OR Logic

The exclusive-OR gate was introduced. Although this circuit is considered a type of logic gate with its own unique symbol, it is actually a combination of two AND gates, one OR gate, and two inverters



(a) Logic diagram



(b) ANSI distinctive
shape symbol

TABLE 5-2

Truth table for an exclusive-OR.

A	B	X
0	0	0
0	1	1
1	0	1
1	1	0

Exclusive-OR logic diagram and symbols

Notice that the output is HIGH only when the two inputs are at opposite levels. A special exclusive-OR operator $\oplus$ is often used, so the expression $X = A\bar{B} + \bar{A}B$ can be stated as “X is equal to A exclusive-OR B” and can be written as

$$X = A \oplus B$$

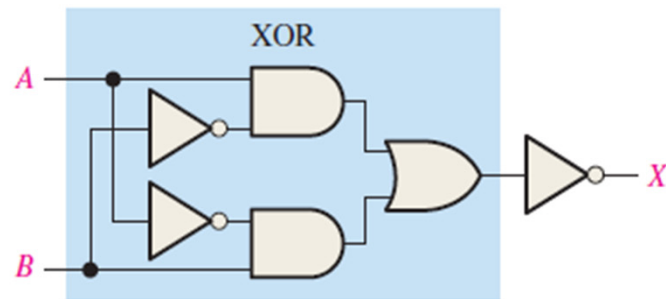
1. Basic Combinational Logic Circuits

Exclusive-NOR Logic

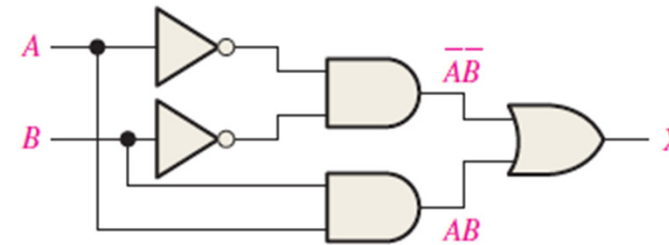
As you know, the complement of the exclusive-OR function is the exclusive-NOR, which is derived as follows

$$X = \overline{AB} + \overline{AB} = (\overline{AB})(\overline{AB}) = (\overline{A} + \overline{B})(A + B) = \overline{A}\overline{B} + AB$$

Notice that the output X is HIGH only when the two inputs, A and B , are at the same level



(a) $X = \overline{AB} + \overline{AB}$



(b) $X = \overline{A}\overline{B} + AB$

Two equivalent ways of implementing the exclusive-NOR

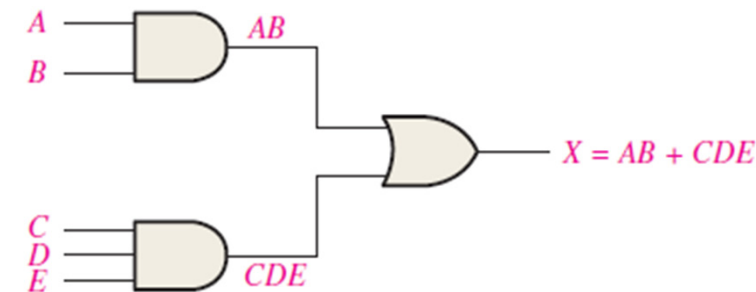
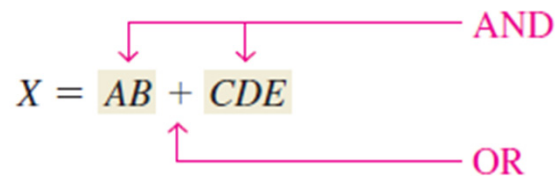
2. Implementing Combinational Logic

Examples are used to illustrate how to implement a logic circuit from a Boolean expression or a truth table

From a Boolean Expression to a Logic Circuit

Let's examine the following Boolean expression:

$$X = AB + CDE$$



Logic circuit

4. Combinational Logic Analysis

2. Implementing Combinational Logic

From a Truth Table to a Logic Circuit

If you begin with a truth table instead of an expression, you can write the SOP expression from the truth table and then implement the logic circuit

TABLE 5-3				
Inputs			Output	Product Term
<i>A</i>	<i>B</i>	<i>C</i>	<i>X</i>	
0	0	0	0	
0	0	1	0	
0	1	0	0	
0	1	1	1	$\overline{A}BC$
1	0	0	1	$A\overline{B}\overline{C}$
1	0	1	0	
1	1	0	0	
1	1	1	0	

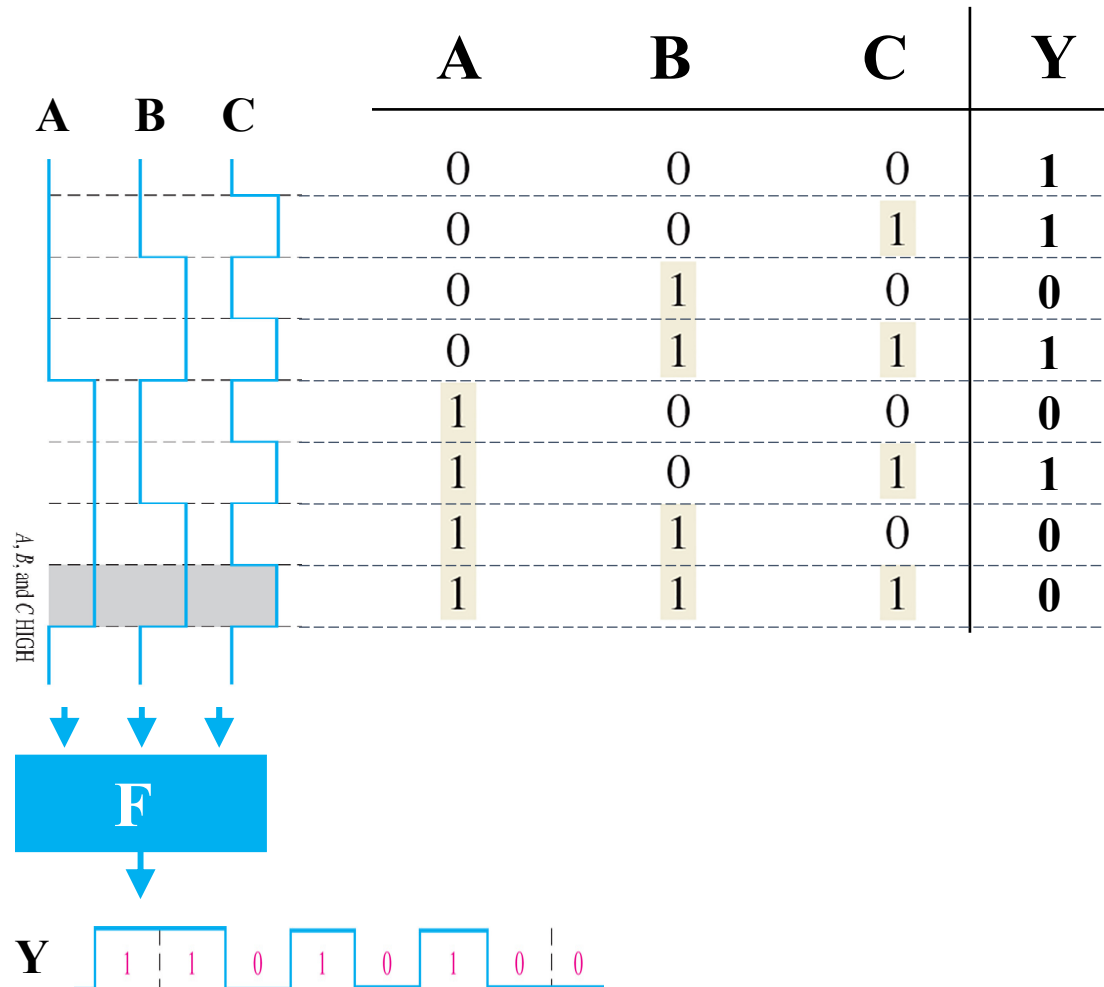
The Boolean SOP expression obtained from the truth table by ORing the product terms for which $X = 1$ is

$$X = \overline{A}BC + A\overline{B}\overline{C}$$

The implementation of this logic function, logic circuit ?

4. Combinational Logic Analysis

(1) 3-input truth table



3-input truth table can describe the input-output relation of a 3-input digital circuit (block F)

The truth table lists all the possible input “combinations” and corresponding outputs, hence **COMBINATIONAL** logic

The truth table can be described using a Boolean expression (function) such as

$$Y = \bar{A}\bar{B} + \bar{A}C + \bar{B}C$$

Corresponding digital circuit can be built using basic logic gates according to Boolean expression

4. Combinational Logic Analysis

(2) 4-input truth table

Truth table because 1 and 0 correspond to TRUE and FALSE in logic

A, B, C, D, and Y are binary numbers also called literals

Y is an output literal related to input literals through a logic function (AND, OR, XOR...): $Y = F(A, B, C, D)$ where F represents a Boolean (logic) function

The values of Y depend on the chosen logic function F and the values of A, B, C, and D

4-input truth table can describe the input-output relation of a 4-input digital circuit:



A	B	C	D	Y
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

4. Combinational Logic Analysis

(3) 4-step design process (truth table $\rightarrow$ circuit)

STEP 1: Capture circuit behavior (input-output relation) in a truth table (skip this step if you can directly write the Boolean expression)

STEP 2: Derive the Boolean expression from the truth table (or directly)

STEP 3: Simplify the Boolean expression (skip this step for now)

STEP 4: Build the circuit using basic logic functions (gates) as stated in the Boolean expression (i.e., draw the gate schematic of the circuit)

4. Combinational Logic Analysis



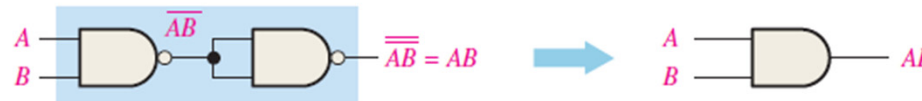
3. The Universal Property of NAND and NOR Gates

The NAND Gate as a Universal Logic Element

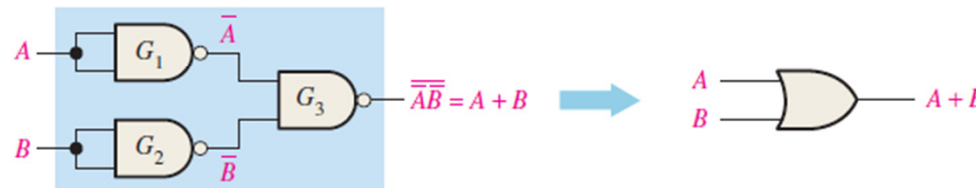
The NAND gate is a **universal gate** because it can be used to produce the NOT, the AND, the OR, and the NOR functions



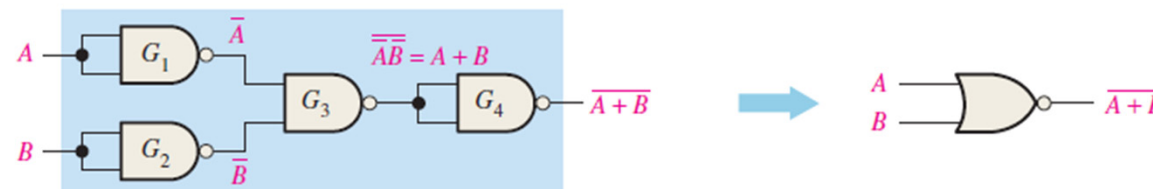
(a) One NAND gate used as an inverter



(b) Two NAND gates used as an AND gate



(c) Three NAND gates used as an OR gate



(d) Four NAND gates used as a NOR gate

4. Combinational Logic Analysis



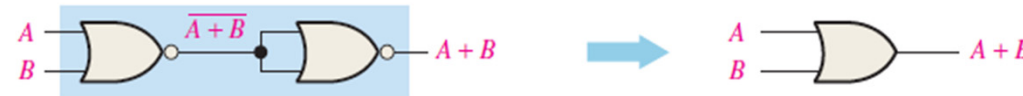
3. The Universal Property of NAND and NOR Gates

The NOR Gate as a Universal Logic Element

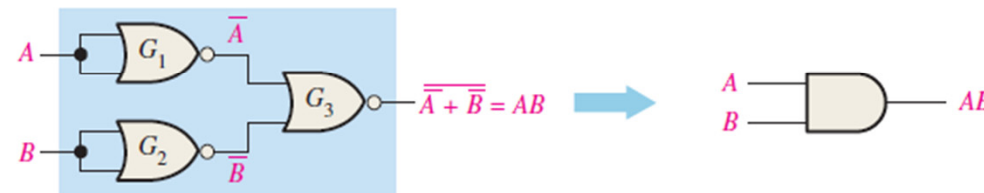
Like the NAND gate, the NOR gate can be used to produce the NOT, AND, OR, and NAND functions



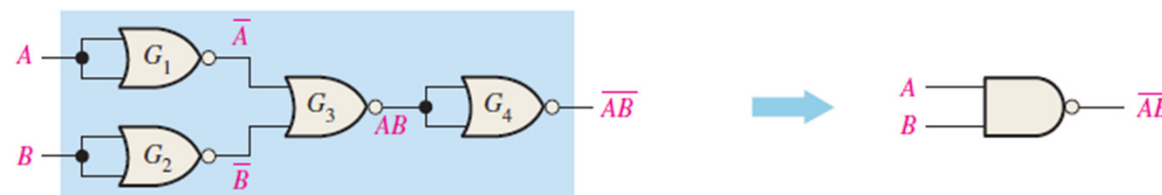
(a) One NOR gate used as an inverter



(b) Two NOR gates used as an OR gate



(c) Three NOR gates used as an AND gate



(d) Four NOR gates used as a NAND gate

Main Content

1. Basic Adders
2. Multi-Bit Adders
3. Optimized Adders

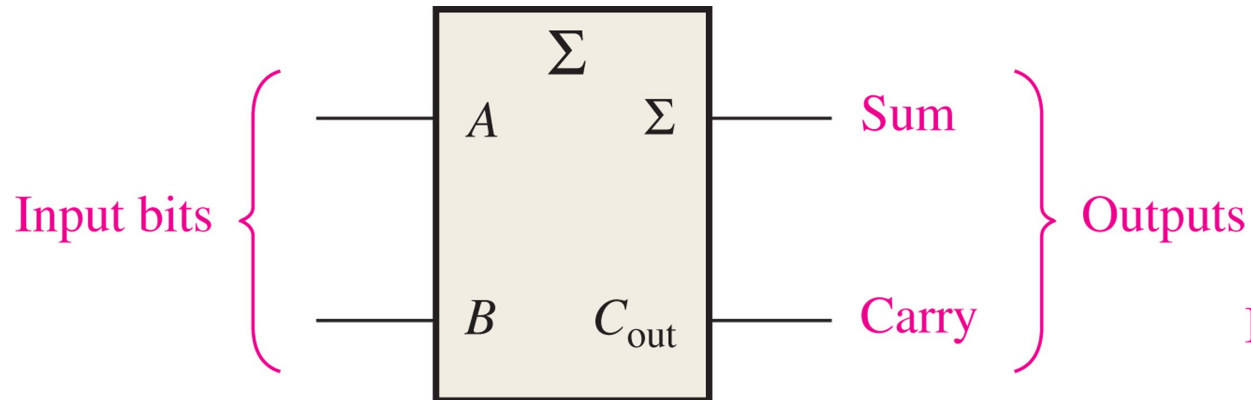
5. Functions of Combinational Logic

1. Basic Adders

(1) Half Adder (truth table, Boolean expression, circuit)

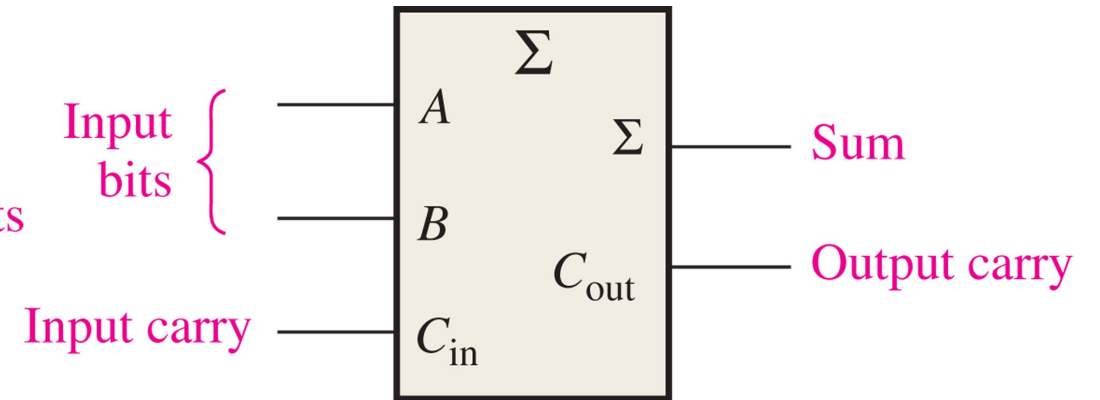
Half Adder circuit:

(no Carry input; adds only two bits)



Full Adder circuit:

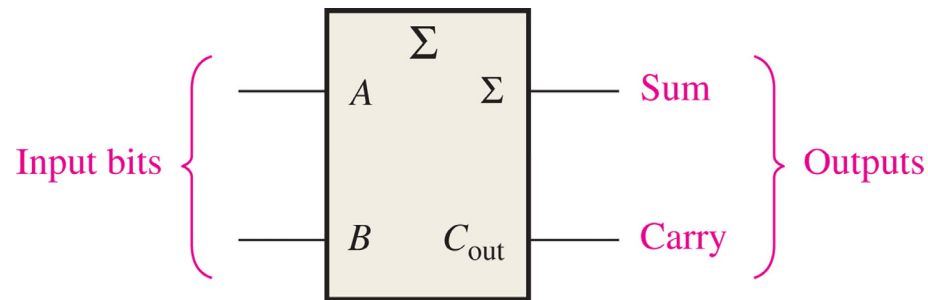
(Carry input; adds three bits)



5. Functions of Combinational Logic

(1) Half Adder (truth table, Boolean expression, circuit)

STEP 1: Capture circuit behavior (input-output relation) in a truth table (skip this step if you can directly write the Boolean expression)



STEP 2: Derive the Boolean expression from the truth table (or directly)

$$\Sigma = A \oplus B = A\bar{B} + \bar{A}B$$

$$C_{\text{out}} = AB$$

Half-adder truth table.

A	B	C_{out}	Σ
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Σ = sum

C_{out} = output carry

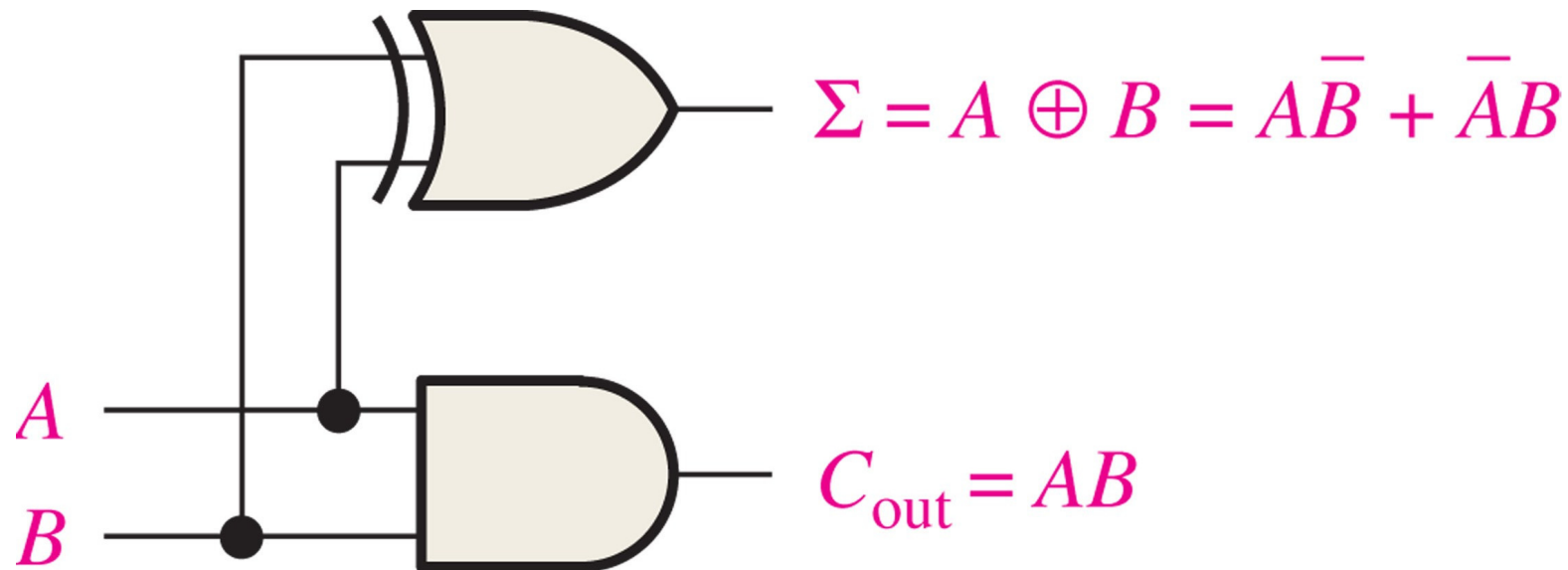
A and B = input variables (operands)

5. Functions of Combinational Logic

(1) Half Adder (truth table, Boolean expression, circuit)

STEP 3: Simplify the Boolean expression (skip this step for now)

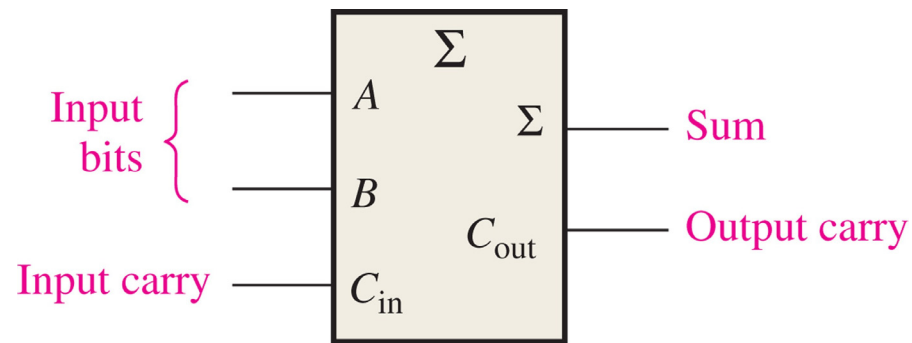
STEP 4: Build the circuit using basic logic functions (gates) as stated in the Boolean expression
(i.e., draw the gate schematic of the circuit)



5. Functions of Combinational Logic

(2) Full Adder (truth table, Boolean expression, cascaded design)

STEP 1: Capture circuit behavior (input-output relation) in a truth table (skip this step if you can directly write the Boolean expression)



STEP 2: Derive the Boolean expression from the truth table (or directly)

$$\Sigma = (A \oplus B) \oplus C_{in}$$

$$C_{out} = AB + (A \oplus B)C_{in}$$

We will see how to derive these Boolean Expressions from the truth table

Full-adder truth table.

A	B	C_{in}	C_{out}	Σ
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

C_{in} = input carry, sometimes designated as CI

C_{out} = output carry, sometimes designated as CO

Σ = sum

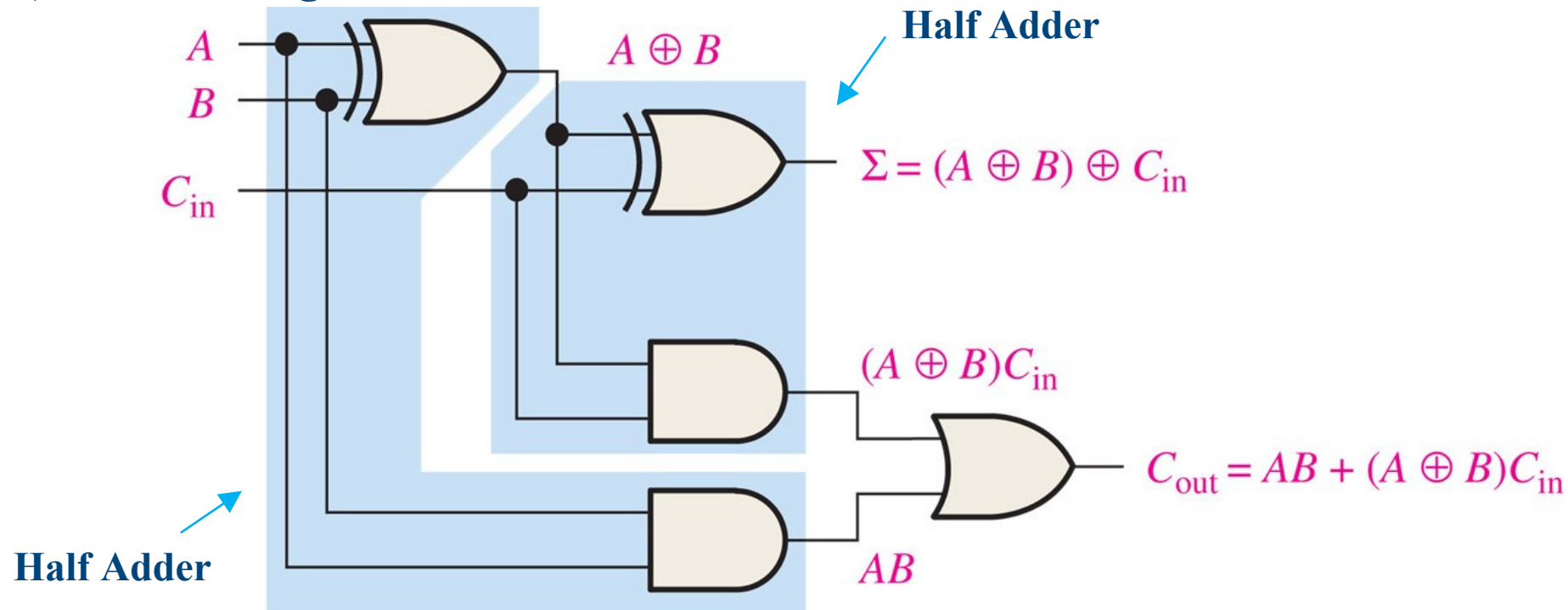
A and B = input variables (operands)

5. Functions of Combinational Logic

(2) Full Adder (truth table, Boolean expression, cascaded design)

STEP 3: Simplify the Boolean expression (skip this step for now)

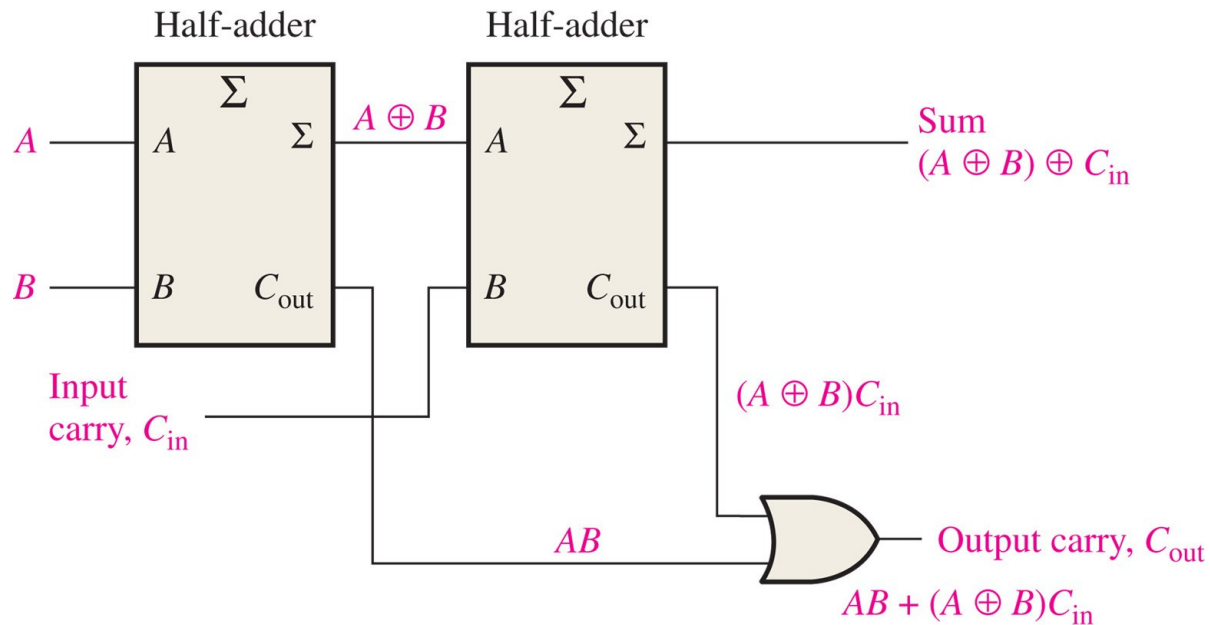
STEP 4: Build the circuit using basic logic functions (gates) as stated in the Boolean expression
(i.e., draw the gate schematic of the circuit)



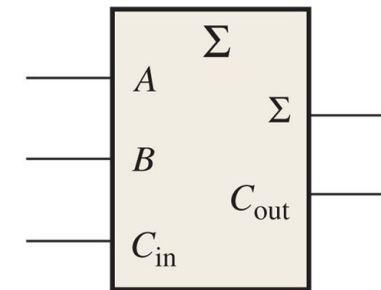
5. Functions of Combinational Logic

(2) Full Adder (truth table, Boolean expression, cascaded design)

Full Adder = 2 Half Adders cascaded and a single two-input OR gate



(a) Arrangement of two half-adders to form a full-adder



(b) Full-adder logic symbol

5. Functions of Combinational Logic

(3) Combinational Logic Design

EXAMPLE: Design a circuit that adds two binary numbers, each having 4 bits

This would require 8-input truth table: $2^8 = 256$ rows!!!

STEP 1: Capture circuit behavior (input-output relationship) in a truth table (skip this step if you can directly write the Boolean expression)

STEP 2: Write the Boolean expression from truth table (or directly)

STEP 3: Simplify the Boolean expression (if possible for now)

STEP 4: Build the circuit using basic logic functions (gates) as stated in the Boolean expression (i.e., draw the gate schematic of the circuit)

NOTE: Truth table is NOT practical for too many inputs (more than four)

For those cases, perform a bit-wise operation (addition) using a known combinational logic block (full adder)

$$\begin{array}{r} A = A_3A_2A_1A_0 \\ + B = B_3B_2B_1B_0 \\ \hline \text{RESULT} = C_{\text{out}}S_3S_2S_1S_0 \end{array}$$

5. Functions of Combinational Logic

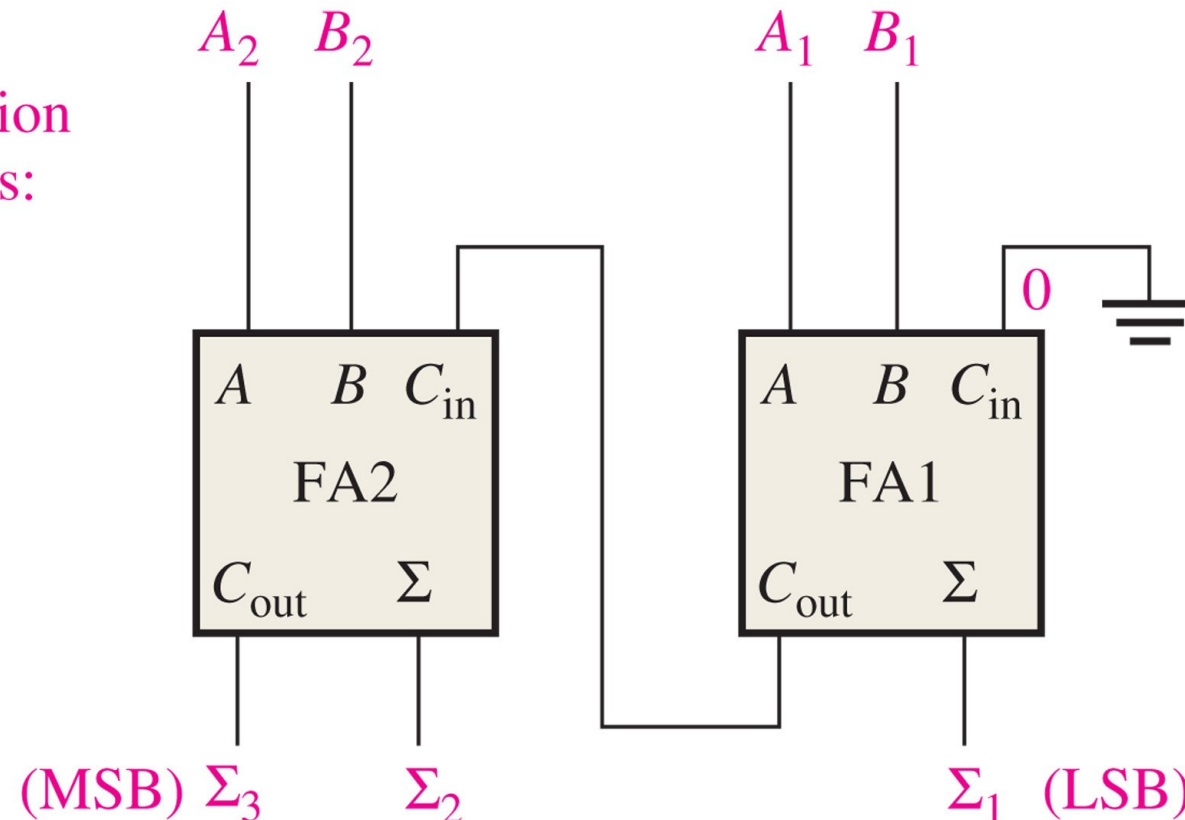
2. Multi-Bit Adders

(1) 2-bit parallel adder

Using 2 Full Adders in parallel configuration (bit-wise addition)

General format, addition
of two 2-bit numbers:

$$\begin{array}{r} A_2A_1 \\ + B_2B_1 \\ \hline \Sigma_3\Sigma_2\Sigma_1 \end{array}$$



THE HONG KONG
UNIVERSITY OF SCIENCE
AND TECHNOLOGY

because carry ripples from a lower stage to a higher stage

The diagram illustrates a 4-bit ripple-carry adder implemented using four 1-bit Full Adders (FA1, FA2, FA3, FA4). The adders are connected in series, with the carry-out of one stage becoming the carry-in of the next. The inputs are $A_4, B_4, A_3, B_3, A_2, B_2, A_1, B_1$. The outputs are $C_4, \Sigma_4, C_3, \Sigma_3, C_2, \Sigma_2, C_1, \Sigma_1$. The carry-in for FA1 is C_0 (LSB). The final carry-out is C_4 (MSB).

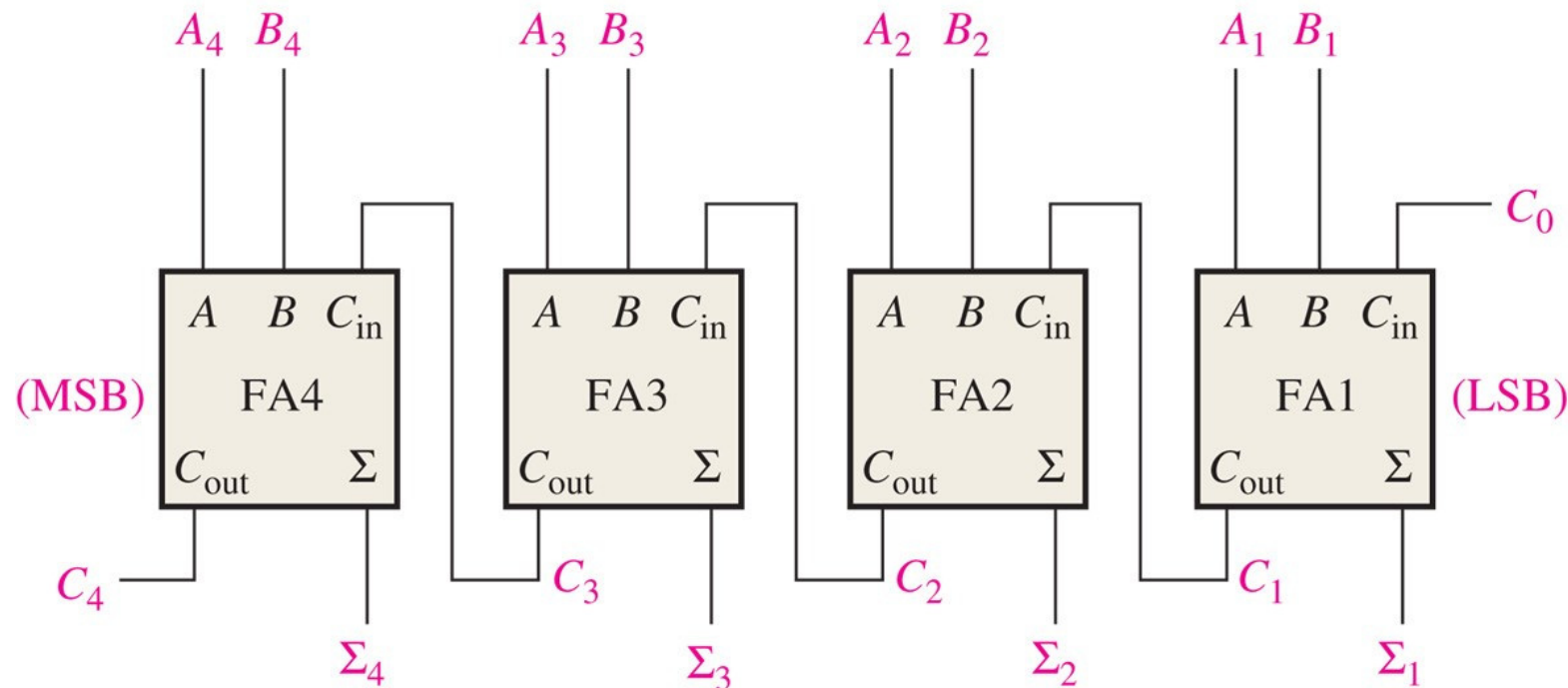
57

5. Functions of Combinational Logic

(2) 4-bit Ripple Carry Adder (structure, delays)

At any stage, the sum and the carry output are not finalized until the input carry arrives, causing a time delay in the process. Imagine 128-bit ripple carry adder!

a stage is one full adder



(a) Block diagram

5. Functions of Combinational Logic

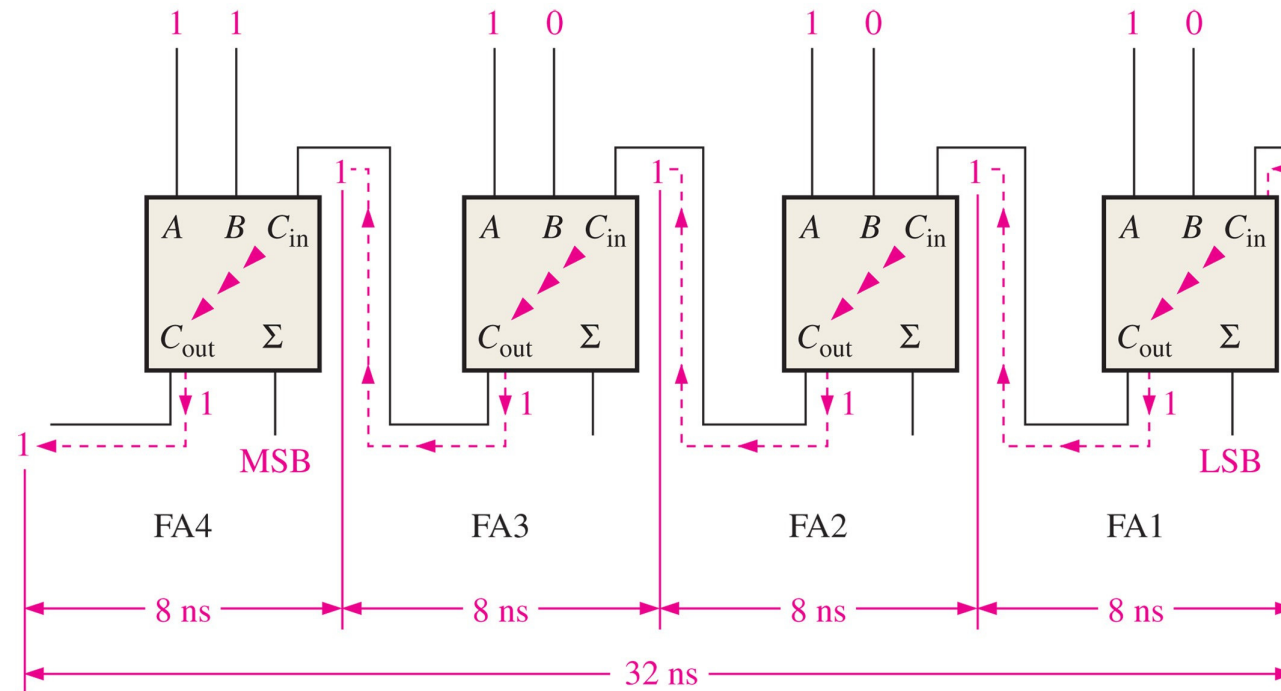
(2) 4-bit Ripple Carry Adder (structure, delays)

Finalized only after the carry input to the LSB stage rippled through all the adders.

The cumulative delay through all the adders is a “worst-case” addition time (32 ns here).

The carry propagation delay for each full-adder:

the time from the application of the input carry until the output carry occurs, assuming that the A and B inputs are already applied.



3. Optimized Adders

(1) Look-Ahead Carry Adder (carry generation/propagation, fast addition)

- The Look-Ahead Carry Adder is a FAST adder
- It speeds up the addition by eliminating the ripple carry delay
- It anticipates the output carry of each stage
- It produces the output carry based on the applied inputs using either
 - Carry generation (C_g) or
 - Carry propagation (C_p)

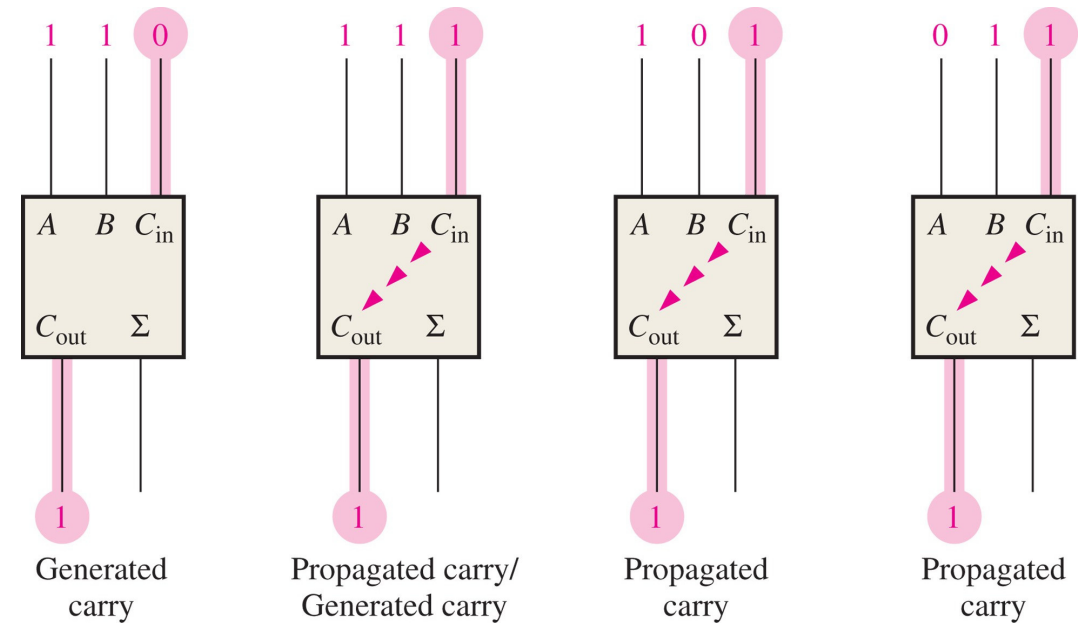
5. Functions of Combinational Logic

(1) Look-Ahead Carry Adder

Carry generation: $C_g = AB$
an output carry is generated by a full adder only when both inputs are 1

Carry propagation: $C_p = A + B$
the input carry is rippled (propagated) to become the output carry.
This occurs when either or both of the input bits are 1s
Hence, a carry is either freshly generated at that stage and/or an incoming carry being propagated

$$C_{out} = C_g + C_p C_{in}$$

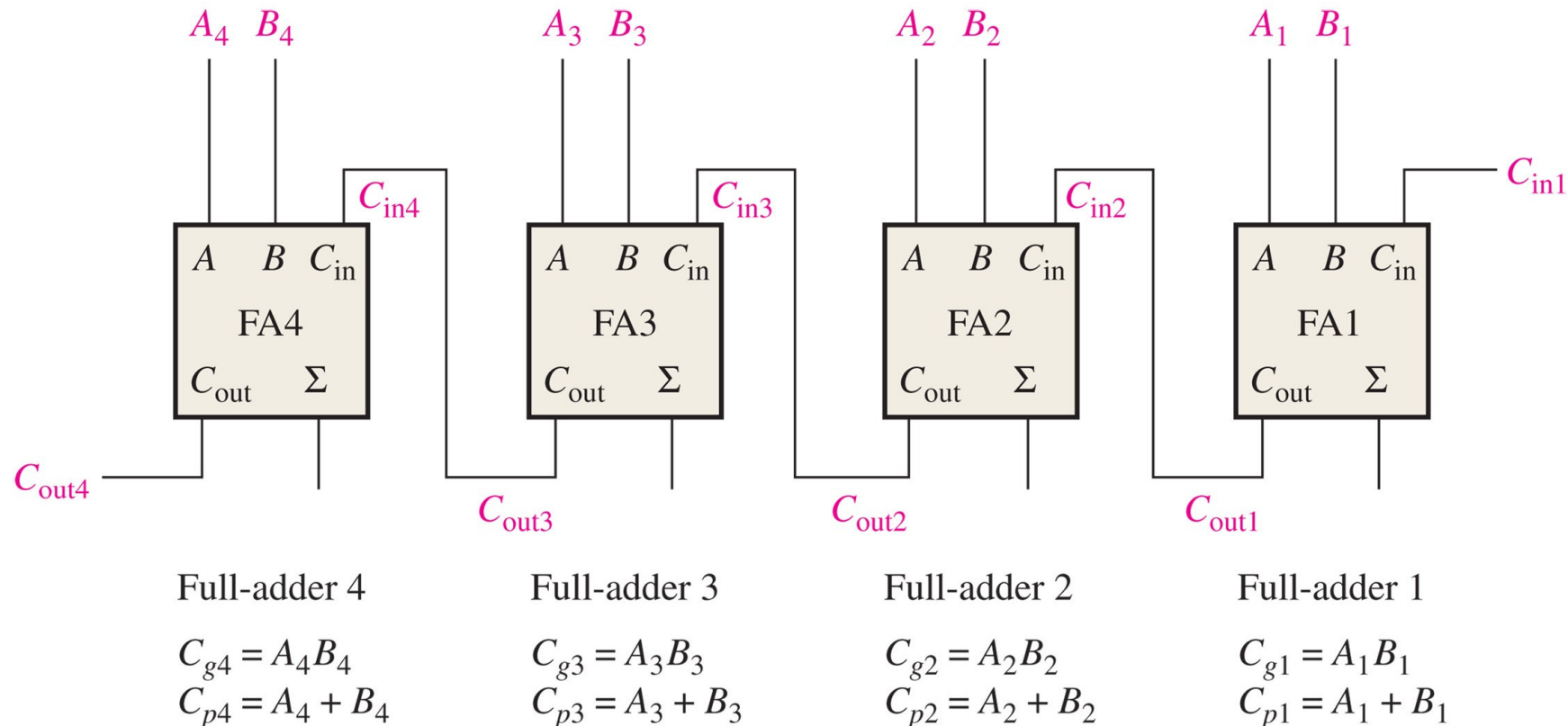


5. Functions of Combinational Logic



(1) Look-Ahead Carry Adder

$$C_{out} = C_g + C_p C_{in}$$



5. Functions of Combinational Logic

(1) Look-Ahead Carry Adder

$$C_{out} = C_g + C_p C_{in}$$

Full-adder 1:

$$C_{out1} = C_{g1} + C_{p1}C_{in1}$$

Full-adder 2:

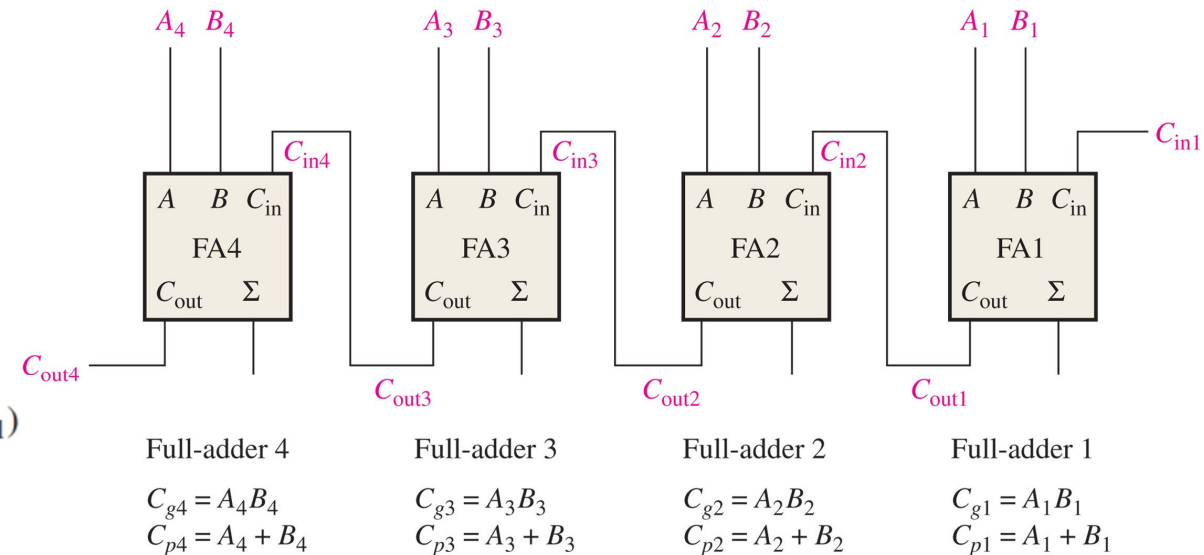
$$\begin{aligned} C_{in2} &= C_{out1} \\ C_{out2} &= C_{g2} + C_{p2}C_{in2} = C_{g2} + C_{p2}C_{out1} = C_{g2} + C_{p2}(C_{g1} + C_{p1}C_{in1}) \\ &= C_{g2} + C_{p2}C_{g1} + C_{p2}C_{p1}C_{in1} \end{aligned}$$

Full-adder 3:

$$\begin{aligned} C_{in3} &= C_{out2} \\ C_{out3} &= C_{g3} + C_{p3}C_{in3} = C_{g3} + C_{p3}C_{out2} = C_{g3} + C_{p3}(C_{g2} + C_{p2}C_{g1} + C_{p2}C_{p1}C_{in1}) \\ &= C_{g3} + C_{p3}C_{g2} + C_{p3}C_{p2}C_{g1} + C_{p3}C_{p2}C_{p1}C_{in1} \end{aligned}$$

Full-adder 4:

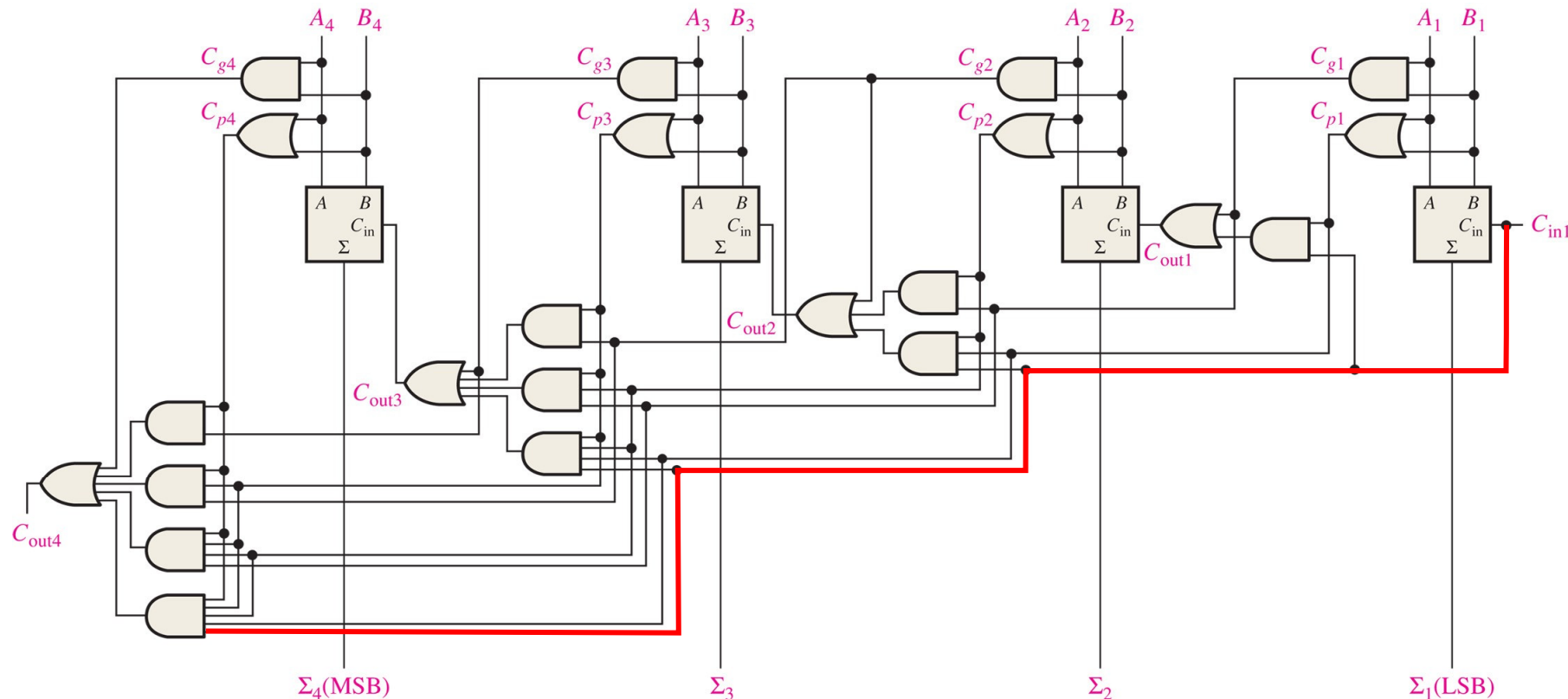
$$\begin{aligned} C_{in4} &= C_{out3} \\ C_{out4} &= C_{g4} + C_{p4}C_{in4} = C_{g4} + C_{p4}C_{out3} \\ &= C_{g4} + C_{p4}(C_{g3} + C_{p3}C_{g2} + C_{p3}C_{p2}C_{g1} + C_{p3}C_{p2}C_{p1}C_{in1}) \\ &= C_{g4} + C_{p4}C_{g3} + C_{p4}C_{p3}C_{g2} + C_{p4}C_{p3}C_{p2}C_{g1} + C_{p4}C_{p3}C_{p2}C_{p1}C_{in1} \end{aligned}$$



5. Functions of Combinational Logic

(1) Look-Ahead Carry Adder 4-bit

- All C_g and C_p values are available after a single-gate delay and all C_{out} values are available after a three-gate delay once all inputs and C_{in1} are applied.
- Carry is independently determined at each stage based on C_g and C_p values and C_{in1}
- So carry does not have to propagate (ripple) from stage to stage



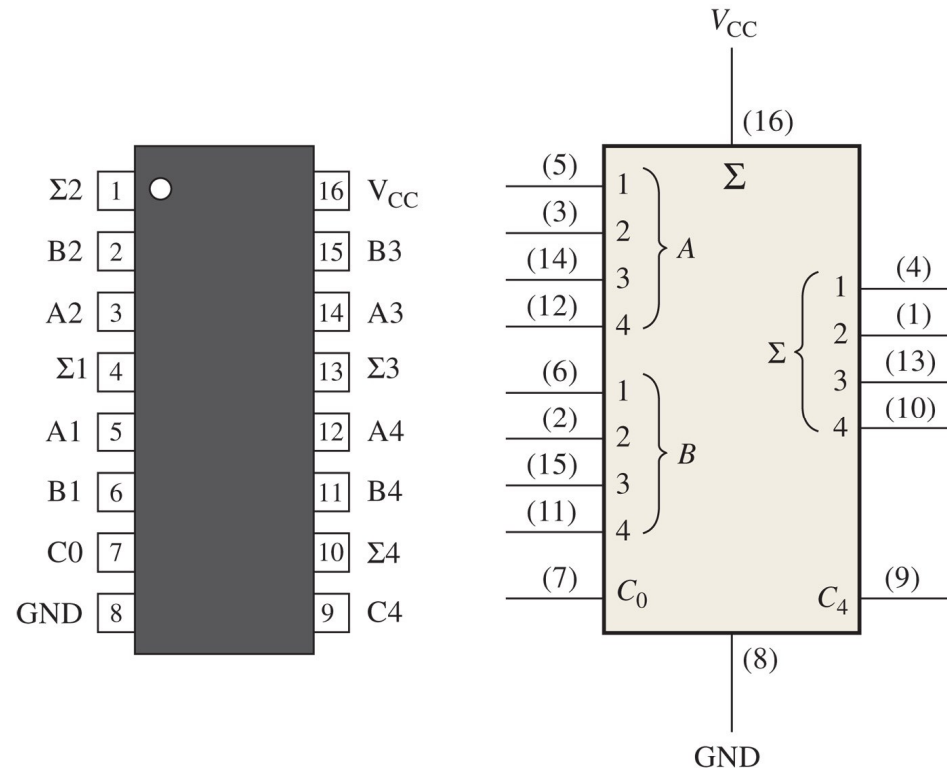
5. Functions of Combinational Logic

(2) 74HC283 IC implementation

The 74HC283/74LS283 4-bit binary full adders with fast carry (with internal carry lookahead).

Fixed-function IC chips

It can be also implemented through VHDL programmed into an FPGA

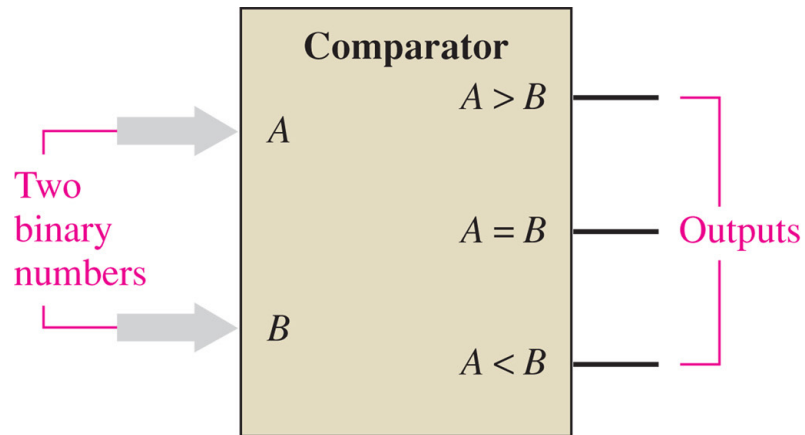


(a) Pin diagram

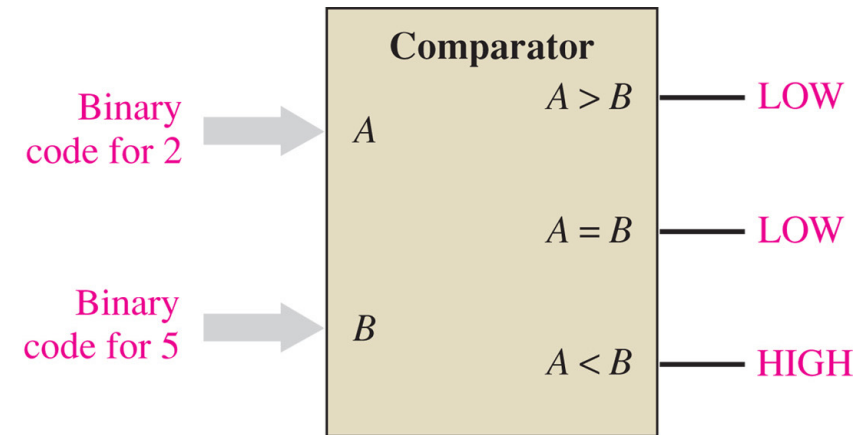
(b) Logic symbol

4. Comparator Basics

(1) Magnitude comparator (truth table, Boolean expressions)



(a) Basic magnitude comparator

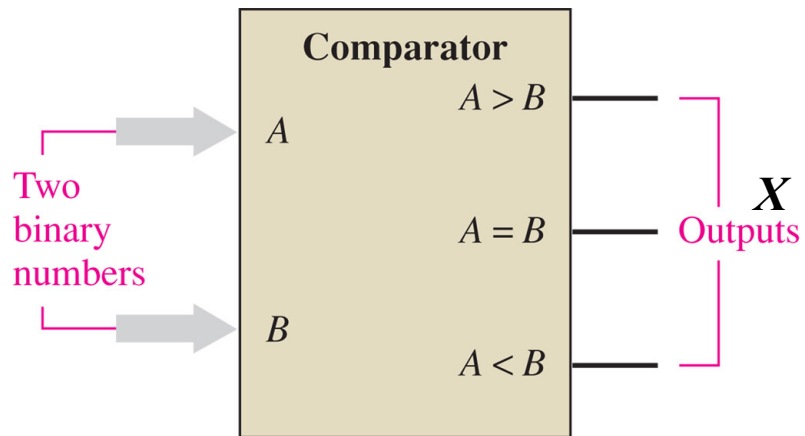


(b) Example: A is less than B ($2 < 5$) as indicated by the HIGH output ($A < B$)

5. Functions of Combinational Logic

(2) Comparator: Simplest case

A and B each 1-bit number
Truth table for each output:

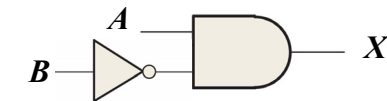


(a) Basic magnitude comparator

A	B	$A > B$
0	0	0
0	1	0
1	0	1
1	1	0

Boolean Expression

$$X = A \bar{B}$$



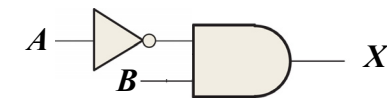
A	B	$A = B$
0	0	1
0	1	0
1	0	0
1	1	1

$$X = \bar{A} \bar{B} + A B$$



A	B	$A < B$
0	0	0
0	1	1
1	0	0
1	1	0

$$X = \bar{A} B$$



6. Logic Functions & Gates



1. Universal Gates

(1) NAND/NOR as universal gates

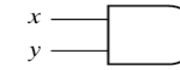
Primitive Gates

- CMOS building blocks
 - Universal Gates
 - Equivalent Gate: 2-input NAND
2-input NOR
- Used to compare complexity of circuits

Inequality Gate/ODD Function

Equality Gate/Even Function

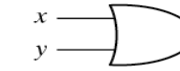
AND



$$F = xy$$

x	y	F
0	0	0
0	1	0
1	0	0
1	1	1

OR



$$F = x + y$$

x	y	F
0	0	0
0	1	1
1	0	1
1	1	1

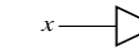
Inverter



$$F = x'$$

x	F
0	1
1	0

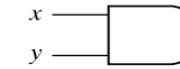
Buffer



$$F = x$$

x	F
0	0
1	1

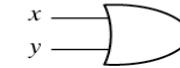
NAND



$$F = (xy)'$$

x	y	F
0	0	1
0	1	1
1	0	1
1	1	0

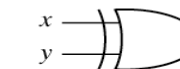
NOR



$$F = (x + y)'$$

x	y	F
0	0	1
0	1	0
1	0	0
1	1	0

Exclusive-OR
(XOR)

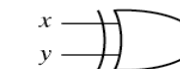


$$F = xy' + x'y$$

$$= x \oplus y$$

x	y	F
0	0	0
0	1	1
1	0	1
1	1	0

Exclusive-NOR
or
equivalence



$$F = xy + x'y'$$

$$= (x \oplus y)'$$

x	y	F
0	0	1
0	1	0
1	0	0
1	1	1

2. 2-Input Functions

(1) All 16 possible 2-input logic functions

Total 4 input combinations
(00,01,10,11)

✓ 10 out of 16
Introduced already
Plus 4 wires

Total 2 output states (0,1) for
each input combination

I N P U T A B																
	Z	✓	✓	Wire to input A	✓	Wire to input B	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	E	A	A	A	B	X	O	O	O	O	O	O	O	O	O	O
	R	N	>	↓	>	↓	R	R	R	R	R	R	R	R	R	R
	O	D	B	A	A	B	R	R	R	R	'B'	B	'A'	A	D	E
00	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
01	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
10	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
11	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1

Plus 2 new functions

Wire
to Ground

Wire
to SUPPLY (V_{DD})

(2) Boolean expressions for XOR/XNOR

Boolean Expressions for the 16 Functions of Two Variables

Boolean Functions	Operator Symbol	Name	Comments
$F_0 = 0$		Null	Binary constant 0
$F_1 = xy$	$x \cdot y$	AND	x and y
$F_2 = xy'$	x/y	Inhibition	x , but not y
$F_3 = x$		Transfer	x
$F_4 = x'y$	y/x	Inhibition	y , but not x
$F_5 = y$		Transfer	y
$F_6 = xy' + x'y$	$x \oplus y$	Exclusive-OR	x or y , but not both
$F_7 = x + y$	$x + y$	OR	x or y
$F_8 = (x + y)'$	$x \downarrow y$	NOR	Not-OR
$F_9 = xy + x'y'$	$(x \oplus y)'$	Equivalence	x equals y
$F_{10} = y'$	y'	Complement	Not y
$F_{11} = x + y'$	$x \subset y$	Implication	If y , then x
$F_{12} = x'$	x'	Complement	Not x
$F_{13} = x' + y$	$x \supset y$	Implication	If x , then y
$F_{14} = (xy)'$	$x \uparrow y$	NAND	Not-AND
$F_{15} = 1$		Identity	Binary constant 1

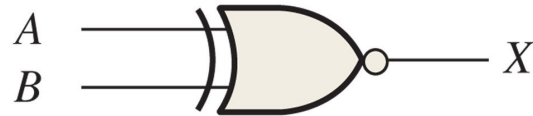
Main Content

1. SOP & POS Methods
2. Full Adder Boolean Analysis
3. Rules & Simplification

7.1 Sum-of-Products (SOP) and Product-of-Sums (POS) Derivation



Example: XNOR function



Two methods

A	B	A = B
0	0	1 SOP
0	1	0 POS
1	0	0 POS
1	1	1 SOP

1. Sum of Products (SOP)

- Identify rows where output values are 1s

Sum refers to OR operation

Product refers to AND operation

- For those, form product terms: AND all literals (negate those with 0s)

- OR all product terms

$$X = \underbrace{\bar{A}\bar{B}}_{\text{1st row where A and B both 0}} + \underbrace{AB}_{\text{Last row where A and B both 1}}$$

2. Product of Sums (POS)

- Identify rows where output values are 0s

- Form a sum term: OR all literals (negate those with 1s)

- AND all sum terms

$$X = \underbrace{(A + \bar{B})}_{\substack{\text{2nd row} \\ A=0 \\ B=1}} \underbrace{(\bar{A} + B)}_{\substack{\text{3rd row} \\ A=1 \\ B=0}}$$

Both expressions
are identical. Proof:
use the distributive
law of Boolean
Algebra

7.1 Sum-of-Products (SOP) and Product-of-Sums (POS) Derivation



Example: XOR function



A	B	$A \neq B$	
0	0	0	POS
0	1	1	SOP
1	0	1	SOP
1	1	0	POS

Two methods

1. Sum of Products (SOP)

- Identify rows where output values are 1s

Sum refers to OR operation

- Form a product term: AND all literals (negate those with 0s)

Product refers to AND operation

- OR all product terms

$$X = \underbrace{\bar{A}B}_{2^{\text{nd}} \text{ row}} + \underbrace{A\bar{B}}_{3^{\text{rd}} \text{ row}}$$

$$A = 0 \ B = 1 \quad A = 1 \ B = 0$$

2. Product of Sums (POS)

- Identify rows where output values are 0s

- Form a sum term: OR all literals (negate those with 1s)

- AND all sum terms

$$X = \underbrace{(A + B)}_{1^{\text{st}} \text{ row where } A \text{ and } B \text{ both } 0} \underbrace{(\bar{A} + \bar{B})}_{\text{Last row where } A \text{ and } B \text{ both } 1}$$

$$1^{\text{st}} \text{ row where } A \text{ and } B \text{ both } 0 \quad \text{Last row where } A \text{ and } B \text{ both } 1$$

Both expressions
are identical. Proof:
use the distributive
law of Boolean
Algebra

7.1 Sum-of-Products (SOP) and Product-of-Sums (POS) Derivation



Example: AND function



A	B	X	
0	0	0	POS
0	1	0	POS
1	0	0	POS
1	1	1	SOP

1 = HIGH, 0 = LOW

Two methods

1. Sum of Products (SOP)

- Identify rows where output values are 1s
- Form a product term: AND all literals (negate those with 0s)
- OR all product terms

Sum refers to OR operation

Product refers to AND operation

$$X = \underline{A} \underline{B}$$

*Last row where
A and B both 1*

2. Product of Sums (POS)

- Identify rows where output values are 0s
- Form a sum term: OR all literals (negate those with 1s)
- AND all sum terms

*Both expressions
are identical. Proof:
use the distributive
law of Boolean
Algebra*

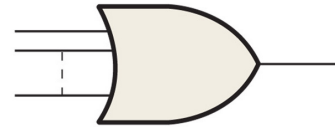
$$X = (\underline{A} + \underline{B})(\underline{A} + \overline{B})(\overline{A} + \underline{B})$$

1st row where A and B both 0 *2nd row A = 0 B = 1* *3rd row A = 1 B = 0*

7.1 Sum-of-Products (SOP) and Product-of-Sums (POS) Derivation



Example: OR function



Two methods

A	B	X	
0	0	0	POS
0	1	1	SOP
1	0	1	SOP
1	1	1	SOP

1 = HIGH, 0 = LOW

Sum refers to OR operation

Product refers to AND operation

1. Sum of Products (SOP)

- Identify rows where output values are 1s
- For those, form product terms: AND all literals (negate those with 0s)
- OR all product terms

$$X = \underbrace{\bar{A}B}_{\substack{2^{\text{nd}} \text{ row} \\ A=0 \ B=1}} + \underbrace{A\bar{B}}_{\substack{3^{\text{rd}} \text{ row} \\ A=1 \ B=0}} + \underbrace{AB}_{\substack{\text{Last row} \\ A=1 \ B=1}}$$

2. Product of Sums (POS)

- Identify rows where output values are 0s
- Form a sum term: OR all literals (negate those with 1s)
- AND all sum terms

$$X = \underbrace{(A + B)}_{\substack{1^{\text{st}} \text{ row where } A \\ \text{and } B \text{ both } 0}}$$

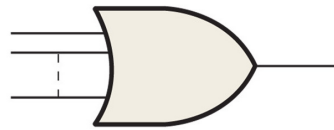
*Both expressions
are identical. Proof:
use Boolean
Algebra*

7.1 Minterms and Maxterms (notation, examples)



<i>A</i>	<i>B</i>	<i>X</i>
0	0	0 POS
0	1	0 POS
1	0	0 POS
1	1	1 SOP

1 = HIGH, 0 = LOW



<i>A</i>	<i>B</i>	<i>X</i>
0	0	0 POS
0	1	1 SOP
1	0	1 SOP
1	1	1 SOP

1 = HIGH, 0 = LOW

Also known as
MINTERMS expansion

Sum of Products

$$X = A B$$

Each product term
is a **MINTERM**

$$X = \bar{A} B + A \bar{B} + AB$$

Also known as
MAXTERMS expansion

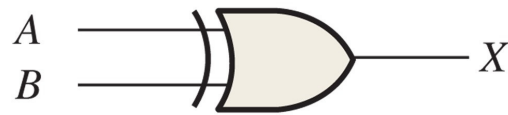
Product of Sums

$$X = (A + B)(A + \bar{B})(\bar{A} + B)$$

Each sum term
is a
MAXTERM

$$X = (A + B)$$

7.1 Minterms and Maxterms (notation, examples)



Exclusive OR (XOR)

Also known as:

Inequality Gate (produces 1 when A and B differs)

A	B	$A \neq B$
0	0	0 POS
0	1	1 SOP
1	0	1 SOP
1	1	0 POS

$$X = \bar{A}B + A\bar{B}$$

$$X = A \oplus B$$

Output is a binary sum of A and B



Exclusive NOR (XNOR)

Also known as:

Equality Gate (produces 1 when A and B same)

A	B	$A = B$
0	0	1 SOP
0	1	0 POS
1	0	0 POS
1	1	1 SOP

$$X = \bar{A}\bar{B} + AB$$

$$X = \overline{A \oplus B}$$

7.1 Minterms and Maxterms (notation, examples)

$$F(x,y,z) = \sum m(1,2,6,7)$$

<i>x</i>	<i>y</i>	<i>z</i>	<i>F</i>	
0	0	0	0	
0	0	1	1	m1
0	1	0	1	m2
0	1	1	0	
1	0	0	0	
1	0	1	0	
1	1	0	1	m6
1	1	1	1	m7

7.1 Minterms and Maxterms (notation, examples)



Minterms are represented by lower-case “m”

Maxterms are represented by capitalized “M”

Table 2.3
Minterms and Maxterms for Three Binary Variables

x	y	z	Minterms		Maxterms	
			Term	Designation	Term	Designation
0	0	0	$x'y'z'$	m_0	$x + y + z$	M_0
0	0	1	$x'y'z$	m_1	$x + y + z'$	M_1
0	1	0	$x'yz'$	m_2	$x + y' + z$	M_2
0	1	1	$x'yz$	m_3	$x + y' + z'$	M_3
1	0	0	$xy'z'$	m_4	$x' + y + z$	M_4
1	0	1	$xy'z$	m_5	$x' + y + z'$	M_5
1	1	0	xyz'	m_6	$x' + y' + z$	M_6
1	1	1	xyz	m_7	$x' + y' + z'$	M_7

7.1 Minterms and Maxterms (notation, examples)



Other notations for SOP and POS as well as minterms and Maxterms

Example: 3-input function

SUM $\swarrow$ minterms

$$F(A,B,C) = \sum (1,4,5,6,7)$$
$$= m1 + m4 + m5 + m6 + m7$$

PRODUCT $\swarrow$ Maxterms

$$F(A,B,C) = \prod (0,2,3)$$
$$= M0 M2 M3$$

A	B	C	F
0	0	0	0 M0
0	0	1	1 m1
0	1	0	0 M2
0	1	1	0 M3
1	0	0	1 m4
1	0	1	1 m5
1	1	0	1 m6
1	1	1	1 m7

7.2 Full adder truth table and XOR/XNOR properties

TABLE 6-2

Full-adder truth table.

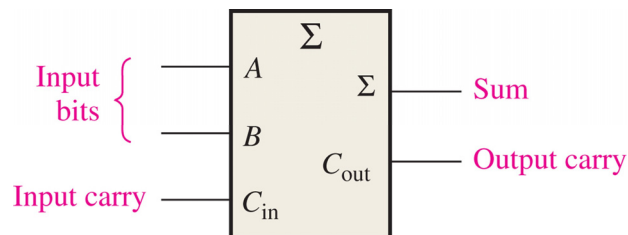
A	B	C_{in}	C_{out}	Σ
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

C_{in} = input carry, sometimes designated as CI

C_{out} = output carry, sometimes designated as CO

Σ = sum

A and B = input variables (operands)



$$C_{out} = \bar{A} B C_{in} + A \bar{B} C_{in} + A B \bar{C}_{in} + A B C_{in}$$



Proof:
Boolean Laws

$$C_{out} = AB + (A \oplus B)C_{in}$$

$$\Sigma = \bar{A} \bar{B} C_{in} + \bar{A} B \bar{C}_{in} + A \bar{B} \bar{C}_{in} + A B C_{in}$$



Proof:
Boolean Laws

$$\Sigma = (A \oplus B) \oplus C_{in}$$

- **XOR** (inequality gate) output can be calculated by simply summing all its input bits
- **XOR** is **ODD** function; it gives an output 1 when an odd number of input bits are 1s
- **XNOR** (equality gate) is **EVEN** function; it gives an output 1 when an even number of input bits are 1s

7.3 Basic Boolean algebra rules

Basic rules of Boolean algebra.

1. $A + 0 = A$

2. $A + 1 = 1$

3. $A \cdot 0 = 0$

4. $A \cdot 1 = A$

5. $A + A = A$

6. $A + \bar{A} = 1$

7. $A \cdot A = A$

8. $A \cdot \bar{A} = 0$

9. $\bar{\bar{A}} = A$

10. $A + AB = A$

11. $A + \bar{A}B = A + B$

12. $(A + B)(A + C) = A + BC$

A , B , or C can represent a single variable or a combination of variables.



香港科技大學
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

Thank you for listening !

